On input symbol $<$ the set has 1

A. a shift-reduce conflict and a reduce-reduce conflict. 2
B. a shift-reduce conflict but not a reduce-reduce conflict.
C. a reduce-reduce conflict but not a shift-reduce conflict.
D. neither a shift-reduce nor a reduce-reduce conflict.

gatecse-2014-set1 compiler-design parsing normal lr-parser 3

Answer key

2.14.10 LR Parser: GATE CSE 2015 | Set 3 | Question: 16

Among simple LR (SLR), canonical LR, and look-ahead LR (LALR), which of the following pairs identify the 4 method that is very easy to implement and the method that is the most powerful, in that order?

A. SLR, LALR
B. Canonical LR, LALR
C. SLR, canonical LR
D. LALR, canonical LR

gatecse-2015-set3 compiler-design parsing normal lr-parser 5

Answer key

2.14.11 LR Parser: GATE CSE 2017 | Set 2 | Question: 6

Which of the following statements about parser is/are CORRECT? 6

I. Canonical LR is more powerful than SLR 7
II. SLR is more powerful than LALR
III. SLR is more powerful than Canonical LR

A. I only B. II only C. III only D. II and III only 8

gatecse-2017-set2 compiler-design parsing lr-parser 9

Answer key

2.14.12 LR Parser: GATE CSE 2019 | Question: 3

Which one of the following kinds of derivation is used by LR parsers? 10

A. Leftmost
B. Leftmost in reverse 11
C. Rightmost
D. Rightmost in reverse

gatecse-2019 compiler-design parsing one-mark lr-parser 12

Answer key

2.14.13 LR Parser: GATE CSE 2020 | Question: 24

Consider the following grammar. 13

- $S \rightarrow aSB \mid d$ 14
- $B \rightarrow b$

The number of reduction steps taken by a bottom-up parser while accepting the string $aaadbbb$ is _____. 15

gatecse-2020 numerical-answers compiler-design lr-parser one-mark 16

Answer key

2.14.14 LR Parser: GATE CSE 2021 | Set 1 | Question: 5

Consider the following statements.

- S_1 : Every $\text{SLR}(1)$ grammar is unambiguous but there are certain unambiguous grammars that are not $\text{SLR}(1)$.
- S_2 : For any context-free grammar, there is a parser that takes at most $O(n^3)$ time to parse a string of length n.

Which one of the following options is correct?

A. S_1 is true and S_2 is false
B. S_1 is false and S_2 is true 1
C. S_1 is true and S_2 is true
D. S_1 is false and S_2 is false

gatecse-2021-set1 compiler-design lr-parser one-mark 2

Answer key

2.14.15 LR Parser: GATE CSE 2021 | Set 2 | Question: 51

Consider the following augmented grammar with $\{\#, @, <, >, a, b, c\}$ as the set of terminals. 3

$$
\begin{aligned}
&S' \rightarrow S \\
&S \rightarrow S\#cS \\
&S \rightarrow SS \\
&S \rightarrow S@ \\
&S \rightarrow < S > \\
&S \rightarrow a \\
&S \rightarrow b \\
&S \rightarrow c
\end{aligned}
$$
4

Let $I_0 = \text{CLOSURE}(\{S' \rightarrow \bullet S\})$. The number of items in the set $\text{GOTO}(\text{GOTO}(I_0, <), <)$ is _____ 5

gatecse-2021-set2 compiler-design lr-parser numerical-answers two-marks 6

Answer key

2.14.16 LR Parser: GATE CSE 2022 | Question: 19

Consider the augmented grammar with $\{+, *, (,), \text{id}\}$ as the set of terminals. 7

- $S' \rightarrow S$
- $S \rightarrow S + R \mid R$
- $R \rightarrow R * P \mid P$
- $P \rightarrow (S) \mid \text{id}$

8

If I_0 is the set of two $LR(0)$ items $\{[S' \rightarrow S.], [S \rightarrow S. + R]\}$, then $goto(closure(I_0), +)$ contains exactly _____ items. 9

gatecse-2022 numerical-answers compiler-design parsing lr-parser one-mark 10

Answer key

2.14.17 LR Parser: GATE CSE 2022 | Question: 3

Which one of the following statements is TRUE? 11

A. The $LALR(1)$ parser for a grammar G cannot have reduce-reduce conflict if the $LR(1)$ parser for G does not have reduce-reduce conflict.
B. Symbol table is accessed only during the lexical analysis phase.
C. Data flow analysis is necessary for run-time memory management.
D. $LR(1)$ parsing is sufficient for deterministic context-free languages.

12

gatecse-2022 compiler-design parsing one-mark lr-parser

Answer key

2.14.18 LR Parser: GATE CSE 2024 | Set 2 | Question: 55

Consider the following augmented grammar, which is to be parsed with a SLR parser. The set of terminals is $\{a, b, c, d, \#, @\}$

$$S' \to S$$
$$S \to SS|Aa|bAc|Bc|bBa$$
$$A \to d\#$$
$$B \to @$$ 1

Let $I_0 = \text{CLOSURE}(\{S' \to \bullet S\})$. The number of items in the set $\text{GOTO}(I_0, S)$ is _____. 2

gatecse-2024-set2 numerical-answers compiler-design lr-parser two-marks

Answer key

2.14.19 LR Parser: GATE CSE 2025 | Set 2 | Question: 30

Given a Context-Free Grammar G as follows: 3

$$S \to Aa|bAc|dc \mid bda$$
$$A \to d$$ 4

Which ONE of the following statements is TRUE? 5

A. G is neither $\text{LALR}(1)$ nor $\text{SLR}(1)$ 6
B. G is $\text{CLR}(1)$, not $\text{LALR}(1)$
C. G is $\text{LALR}(1)$, not $\text{SLR}(1)$
D. G is $\text{LALR}(1)$, also $\text{SLR}(1)$

gatecse2025-set2 compiler-design context-free-grammar parsing lr-parser two-marks 7

Answer key

2.14.20 LR Parser: GATE CSE 2026 | Set 2 | Question: 31

Consider the canonical $LR(0)$ parsing of the grammar below using terminals $\{a, b, c\}$ and non-terminals $\{A, B, C, S\}$ with S as the start symbol. 8

$$S \to ACB$$ 9
$$A \to aA \mid \epsilon$$
$$C \to cC \mid \epsilon$$
$$B \to bB \mid b$$

Which one of the following options gives the number of shift-reduce conflicts that will occur in the $LR(0)$ ACTION table? 10

A. 2 B. 3 C. 4 D. 5 11

gatecse-2026-set2 compiler-design lr-parser two-marks 12

Answer key

2.15 Lexical Analysis (6)

✍ **Practice Tests:** Test 1 (10Q) Weekly Quiz 1 (10Q)

2.15.1 Lexical Analysis: GATE CSE 1987 | Question: 1-xvii

Using longer identifiers in a program will necessarily lead to:

A. Somewhat slower compilation
B. A program that is easier to understand

C. An incorrect program D. None of the above 1

gate1987 compiler-design lexical-analysis

Answer key

2.15.2 Lexical Analysis: GATE CSE 2000 | Question: 1.18, ISRO2015-25

The number of tokens in the following C statement is 2

```
printf("i=%d, &i=%x", i, &i);
```
3

A. 3 B. 26 C. 10 D. 21 4

gatecse-2000 compiler-design lexical-analysis easy isro2015 5

Answer key 6

2.15.3 Lexical Analysis: GATE CSE 2010 | Question: 13

Which data structure in a compiler is used for managing information about variables and their attributes? 7

A. Abstract syntax tree B. Symbol table 8
C. Semantic stack D. Parse table

gatecse-2010 compiler-design lexical-analysis easy 9

Answer key 10

2.15.4 Lexical Analysis: GATE CSE 2011 | Question: 1

In a compiler, keywords of a language are recognized during 11

A. parsing of the program B. the code generation 12
C. the lexical analysis of the program D. dataflow analysis

gatecse-2011 compiler-design lexical-analysis easy 13

Answer key

2.15.5 Lexical Analysis: GATE CSE 2011 | Question: 19

The lexical analysis for a modern computer language such as Java needs the power of which one of the following machine models in a necessary and sufficient sense? 14

A. Finite state automata B. Deterministic pushdown automata 15
C. Non-deterministic pushdown automata D. Turing machine

gatecse-2011 compiler-design lexical-analysis easy 16

Answer key 17

2.15.6 Lexical Analysis: GATE CSE 2018 | Question: 37

A lexical analyzer uses the following patterns to recognize three tokens $T_1, T_2,$ and T_3 over the alphabet $\{a, b, c\}$. 18

- $T_1 : a?(b \mid c)^*a$ 19
- $T_2 : b?(a \mid c)^*b$
- $T_3 : c?(b \mid a)^*c$

Note that '$x?$' means 0 or 1 occurrence of the symbol x. Note also that the analyzer outputs the token that matches the longest possible prefix. 20

If the string $bbaacabc$ is processed by the analyzer, which one of the following is the sequence of tokens it outputs? 21

A. $T_1T_2T_3$ B. $T_1T_1T_3$ C. $T_2T_1T_3$ D. T_3T_3 22

gatecse-2018 compiler-design lexical-analysis normal two-marks 23

Answer key

2.16 Linker (3)

2.16.1 Linker: GATE CSE 1991 | Question: 03,ix

A “link editor” is a program that: 1

A. matches the parameters of the macro-definition with locations of the parameters of the macro call 2
B. matches external names of one program with their location in other programs
C. matches the parameters of subroutine definition with the location of parameters of subroutine call.
D. acts as a link between text editor and the user
E. acts as a link between compiler and the user program

gate1991 compiler-design normal linker multiple-selects 3

Answer key

2.16.2 Linker: GATE CSE 2003 | Question: 76

Which of the following is NOT an advantage of using shared, dynamically linked libraries as opposed to using statistically linked libraries? 4

A. Smaller sizes of executable files 5
B. Lesser overall page fault rate in the system
C. Faster program startup
D. Existing programs need not be re-linked to take advantage of newer versions of libraries

gatecse-2003 compiler-design runtime-environment linker easy 6

Answer key

2.16.3 Linker: GATE CSE 2004 | Question: 9

Consider a program P that consists of two source modules M_1 and M_2 contained in two different files. If M_1 contains a reference to a function defined in M_2 the reference will be resolved at 7 8

A. Edit time B. Compile time C. Link time D. Load time 9

gatecse-2004 compiler-design easy linker 10

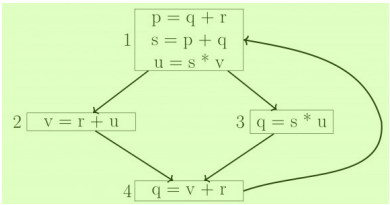
Answer key 11

2.17 Live Variable Analysis (3)

2.17.1 Live Variable Analysis: GATE CSE 2015 | Set 1 | Question: 50

A variable x is said to be live at a statement s_i in a program if the following three conditions hold simultaneously: 12 13

i. There exists a statement S_j that uses x 14
ii. There is a path from S_i to S_j in the flow graph corresponding to the program
iii. The path has no intervening assignment to x including at S_i and S_j



15

The variables which are live both at the statement in basic block 2 and at the statement in basic block 3 of the above control flow graph are 16

A. p, s, u B. r, s, u C. r, u D. q, v 17

gatecse-2015-set1 compiler-design live-variable-analysis normal 18

Answer key

2.17.2 Live Variable Analysis: GATE CSE 2021 | Set 2 | Question: 38

For a statement S in a program, in the context of liveness analysis, the following sets are defined: 1

$\mathrm{USE}(S)$: the set of variables used in S
$\mathrm{IN}(S)$: the set of variables that are live at the entry of S
$\mathrm{OUT}(S)$: the set of variables that are live at the exit of S 2

Consider a basic block that consists of two statements, S_1 followed by S_2. Which one of the following statements is correct? 3

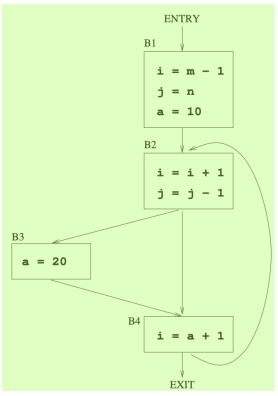
A. $\mathrm{OUT}(S_1) = \mathrm{IN}(S_2)$
B. $\mathrm{OUT}(S_1) = \mathrm{IN}(S_1) \cup \mathrm{USE}(S_1)$
C. $\mathrm{OUT}(S_1) = \mathrm{IN}(S_2) \cup \mathrm{OUT}(S_2)$
D. $\mathrm{OUT}(S_1) = \mathrm{USE}(S_1) \cup \mathrm{IN}(S_2)$ 4

gatecse-2021-set2 code-optimization live-variable-analysis compiler-design two-marks

Answer key

2.17.3 Live Variable Analysis: GATE CSE 2023 | Question: 27

Consider the control flow graph shown. 5



6

Which one of the following choices correctly lists the set of *live* variables at the exit point of each basic block? 7

A. B1: { }, B2: {a}, B3: {a}, B4: {a}
B. B1: {i, j}, B2: {a}, B3: {a}, B4: {i}
C. B1: {a, i, j}, B2: {a, i, j}, B3: {a, i}, B4: {a}
D. B1: {a, i, j}, B2: {a, j}, B3: {a, j}, B4: {a, i, j} 8

gatecse-2023 compiler-design live-variable-analysis two-marks

Answer key

2.18 LI Parser (2)

2.18.1 LI Parser: GATE CSE 2024 | Set 2 | Question: 30

Consider the following context-free grammar where the start symbol is S and the set of terminals is $\{a, b, c, d\}$. 9

$$S \rightarrow AaAb \mid BbBa$$
$$A \rightarrow cS \mid \epsilon$$
$$B \rightarrow dS \mid \epsilon$$
10

The following is a partially-filled $\mathrm{LL}(1)$ parsing table. 11

	a	b	c	d	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	(1)	(2)	
A	$A \rightarrow \varepsilon$	(3)	$A \rightarrow cS$		
B	(4)	$B \rightarrow \varepsilon$		$B \rightarrow dS$	

1

Which one of the following options represents the CORRECT combination for the numbered cells in the parsing table? 2

Note: In the options, "blank" denotes that the corresponding cell is empty. 3

A. $(1)\ S \rightarrow AaAb\ (2)\ S \rightarrow BbBa\ (3)\ A \rightarrow \epsilon\ (4)\ B \rightarrow \epsilon$
B. $(1)\ S \rightarrow BbBa\ (2)\ S \rightarrow AaAb\ (3)\ A \rightarrow \epsilon\ (4)\ B \rightarrow \epsilon$
C. $(1)\ S \rightarrow AaAb\ (2)\ S \rightarrow BbBa\ (3)$ blank (4) blank
D. $(1)\ S \rightarrow BbBa\ (2)\ S \rightarrow AaAb\ (3)$ blank (4) blank

4

gatecse-2024-set2 compiler-design ll-parser parsing two-marks

Answer key

2.18.2 LI Parser: GATE CSE 2026 | Set 1 | Question: 18

Which of the following statements is/are true? 5

A. $\mathrm{LL}(1)$ parser uses backtracking
B. For a grammar to be $\mathrm{LL}(1)$, it must be left-recursive
C. For a grammar to be $\mathrm{LL}(1)$, it must be left-factored
D. The $\mathrm{LL}(1)$ parsers are more powerful than the SLR parsers

6

gatecse-2026-set1 compiler-design parsing ll-parser multiple-selects one-mark 7

Answer key

2.19 Macros (4)

2.19.1 Macros: GATE CSE 1992 | Question: 01,vii

Macro expansion is done in pass one instead of pass two in a two pass macro assembler because 8

_____

gate1992 compiler-design macros easy fill-in-the-blanks

Answer key

2.19.2 Macros: GATE CSE 1995 | Question: 1.11

What are x and y in the following macro definition? 9

```
macro Add x, y
      Load y
      Mul x
      Store y
end macro
```

10

A. Variables
B. Identifiers
C. Actual parameters
D. Formal parameters

11

gate1995 compiler-design macros easy 12

Answer key

2.19.3 Macros: GATE CSE 1996 | Question: 2.16

Which of the following macros can put a macro assembler into an infinite loop?

i.
```
.MACRO M1, X
.IF EQ, X   ;if X=0 then
M1 X + 1
```

```
.ENDC
.IF NE, X   ;if X ≠ O then
.WORD X  ;address (X) is stored here
.ENDC
.ENDM
```
1

ii.
```
.MACRO M2, X
.IF EQ, X
M2 X
.ENDC
.IF NE, X
.WORD X + 1
.ENDC
.ENDM
```
2

A. (ii) only B. (i) only C. both (i) and (ii) D. None of the above 3

gate1996 compiler-design macros normal 4

Answer key 5

2.19.4 Macros: GATE CSE 1997 | Question: 1.9

The conditional expansion facility of macro processor is provided to 6

A. test a condition during the execution of the expanded program
B. to expand certain model statements depending upon the value of a condition during the execution of the expanded program
C. to implement recursion
D. to expand certain model statements depending upon the value of a condition during the process of macro expansion

7

gate1997 compiler-design macros easy

Answer key

2.20 Operator Precedence (9)

✍ **Practice Test:** Test 1 (7Q) 8

2.20.1 Operator Precedence: GATE CSE 1991 | Question: 10a

Consider the following grammar for arithmetic expressions using binary operators $-$ and $/$ which are not associative 9

- $E \rightarrow E - T \mid T$ 10
- $T \rightarrow T/F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol) 11

Is the grammar unambiguous? Is so, what is the relative precedence between $-$ and $/$? If not, give an unambiguous grammar that gives $/$ precedence over $-$. 12

gate1991 grammar compiler-design normal descriptive ambiguous-grammar operator-precedence

Answer key

2.20.2 Operator Precedence: GATE CSE 1997 | Question: 1.6

In the following grammar

- $X ::= X \oplus Y \mid Y$
- $Y ::= Z * Y \mid Z$
- $Z ::= id$

Which of the following is true?

A. ‘$\oplus$’ is left associative while ‘$*$’ is right associative 1
B. Both ‘$\oplus$’ and ‘$*$’ are left associative
C. ‘$\oplus$’ is right associative while ‘$*$’ is left associative
D. None of the above

gate1997 compiler-design grammar normal operator-precedence ambiguous-grammar

Answer key

2.20.3 Operator Precedence: GATE CSE 2000 | Question: 2.21, ISRO2015-24

Given the following expression grammar: 2

$$E \rightarrow E * F \mid F + E \mid F$$

$$F \rightarrow F - F \mid id$$ 3

Which of the following is true? 4

A. $*$ has higher precedence than $+$
B. $-$ has higher precedence than $*$ 5
C. $+$ and $-$ have same precedence
D. $+$ has higher precedence than $*$

gatecse-2000 operator-precedence normal compiler-design isro2015 ambiguous-grammar 6

Answer key

2.20.4 Operator Precedence: GATE CSE 2002 | Question: 22

A. Construct all the parse trees corresponding to $i + j * k$ for the grammar
$E \rightarrow E + E$
$E \rightarrow E * E$
$E \rightarrow id$
B. In this grammar, what is the precedence of the two operators $*$ and $+$?
C. If only one parse tree is desired for any string in the same language, what changes are to be made so that the resulting LALR(1) grammar is unambiguous?

7

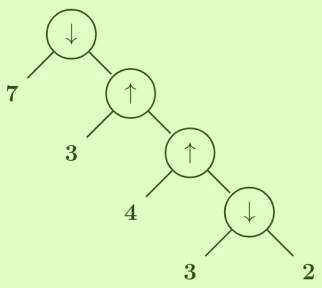
gatecse-2002 compiler-design parsing normal descriptive operator-precedence ambiguous-grammar 8

Answer key

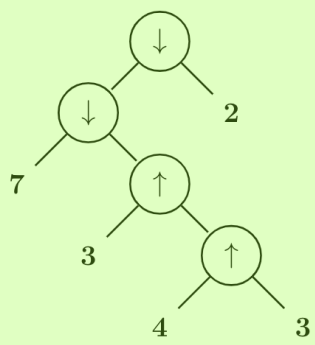
2.20.5 Operator Precedence: GATE CSE 2011 | Question: 27

Consider two binary operators ‘$\uparrow$’ and ‘$\downarrow$’ with the precedence of operator $\downarrow$ being lower than that of the operator $\uparrow$. Operator $\uparrow$ is right associative while operator $\downarrow$ is left associative. Which one of the following represents the parse tree for expression $(7 \downarrow 3 \uparrow 4 \uparrow 3 \downarrow 2)$ 9

A. 10

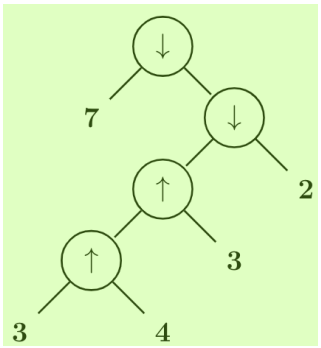


B. 11

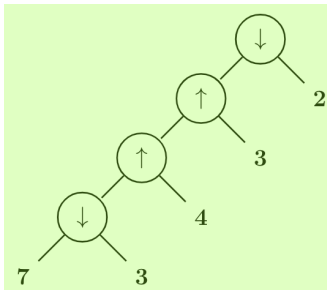


C. 12

D.



1



2

gatecse-2011 compiler-design parsing normal operator-precedence ambiguous-grammar

Answer key

2.20.6 Operator Precedence: GATE CSE 2014 | Set 2 | Question: 17

Consider the grammar defined by the following production rules, with two operators $*$ and $+$ 3

- $S \rightarrow T * P$
- $T \rightarrow U \mid T * U$
- $P \rightarrow Q + P \mid Q$
- $Q \rightarrow Id$
- $U \rightarrow Id$

4

Which one of the following is TRUE? 5

A. $+$ is left associative, while $*$ is right associative 6
B. $+$ is right associative, while $*$ is left associative
C. Both $+$ and $*$ are right associative
D. Both $+$ and $*$ are left associative

gatecse-2014-set2 compiler-design grammar normal operator-precedence ambiguous-grammar 7

Answer key

2.20.7 Operator Precedence: GATE CSE 2016 | Set 1 | Question: 45

The attribute of three arithmetic operators in some programming language are given below. 8

OPERATOR	PRECEDENCE	ASSOCIATIVITY	ARITY
$+$	High	Left	Binary
$-$	Medium	Right	Binary
$*$	Low	Left	Binary

9

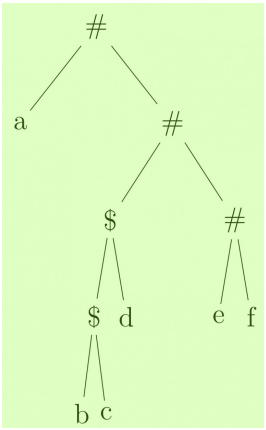
The value of the expression $2 - 5 + 1 - 7 * 3$ in this language is _____. 10

gatecse-2016-set1 compiler-design parsing normal numerical-answers operator-precedence 11

Answer key

2.20.8 Operator Precedence: GATE CSE 2018 | Question: 38

Consider the following parse tree for the expression a#bcd#e#f, involving two binary operators $ and #.



1

Which one of the following is correct for the given parse tree? 2

A. $ has higher precedence and is left associative; # is right associative 3
B. # has higher precedence and is left associative; $ is right associative
C. $ has higher precedence and is left associative; # is left associative
D. $ has higher precedence and is right associative; # is left associative

gatecse-2018 compiler-design parsing normal two-marks operator-precedence ambiguous-grammar

Answer key

2.20.9 Operator Precedence: GATE CSE 2024 | Set 1 | Question: 23

Consider the operator precedence and associativity rules for the *integer* arithmetic operators given in the table below. 4

Operator	Precedence	Associativity
$+$	Highest	Left
$-$	High	Right
$*$	Medium	Right
$/$	Low	Right

5

The value of the expression $3+1+5*2/7+2-4-7-6/2$ as per the above rules is _____. 6

gatecse-2024-set1 numerical-answers compiler-design operator-precedence one-mark

Answer key

2.21 Parameter Passing (14)

✍ **Practice Test:** Test 1 (7Q)

2.21.1 Parameter Passing: GATE CSE 1988 | Question: 2xv

What is printed by following program, assuming call-by reference method of passing parameters for all variables in the parameter list of procedure P?

```
program    Main(inout, output);
var    a, b:integer;
       procedure P(x, y, z:integer);
       begin
          y:=y+1
          z:=x+x
       end P;
begin
   a:=2; b:=3;
   p(a+b, a, a);
   Write(a)
end.
```

gate1988 descriptive compiler-design runtime-environment parameter-passing numerical-answers

Answer key

2.21.2 Parameter Passing: GATE CSE 1988 | Question: 8i

Consider the procedure declaration: 1

```
Procedure
P (k: integer)
```

where the parameter passing mechanism is call-by-value-result. Is it correct if the call, P (A[i]), where A is an array and i an integer, is implemented as below. 2

a. create a new local variable, say z;
b. assign to z, the value of A [i];
c. execute the body of P using z for k;
d. set A [i] to z;
Explain your answer. If this is incorrect implementation, suggest a correct one. 3

gate1988 descriptive compiler-design runtime-environment parameter-passing

Answer key

2.21.3 Parameter Passing: GATE CSE 1989 | Question: 3-viii

In which of the following case(s) is it possible to obtain different results for call-by-reference and call-by-name parameter passing? 4

A. Passing an expression as a parameter
B. Passing an array as a parameter 5
C. Passing a pointer as a parameter
D. Passing as array element as a parameter

gate1989 parameter-passing runtime-environment compiler-design multiple-selects

Answer key

2.21.4 Parameter Passing: GATE CSE 1990 | Question: 11a

What does the following program output? 6

```
program module (input, output);
var
   a:array [1...5] of integer;
   i, j: integer;
procedure unknown (var b: integer, var c: integer);
var
   i:integer;
begin
   for i := 1 to 5 do a[i] := i;
   b:= 0; c := 0
   for i := 1 to 5 do write (a[i]);
   writeln();
   a[3]:=11; a[1]:=11;
   for i:=1 to 5 do a [i] := sqr(a[i]);
   writeln(c,b); b := 5; c := 6;
end;
begin
 i:=1; j:=3; unknown (a[i], a[j]);
 for i:=1 to 5 do write (a[i]);
end;
```

7

gate1990 descriptive compiler-design runtime-environment parameter-passing 8

Answer key 9

2.21.5 Parameter Passing: GATE CSE 1991 | Question: 03,x

Indicate all the true statements from the following:

A. Recursive descent parsing cannot be used for grammar with left recursion.
B. The intermediate form for representing expressions which is best suited for code optimization is the postfix form.
C. A programming language not supporting either recursion or pointer type does not need the support of dynamic memory allocation.

D. Although C does not support call-by-name parameter passing, the effect can be correctly simulated in C 1
E. No feature of Pascal typing violates strong typing in Pascal.

gate1991 compiler-design parameter-passing difficult multiple-selects

Answer key

2.21.6 Parameter Passing: GATE CSE 1991 | Question: 09a

Consider the following pseudo-code (all data items are of type integer): 2

```
procedure P(a, b, c);
   a := 2;
   c := a + b;
end {P}

begin
   x := 1;
   y := 5;
   z := 100;
   P(x, x*y, z);
   Write ('x = ', x, 'z = ', z);
end
```

3

Determine its output, if the parameters are passed to the Procedure P by 4

i. value 5
ii. reference
iii. name

gate1991 compiler-design parameter-passing normal runtime-environment descriptive 6

Answer key

2.21.7 Parameter Passing: GATE CSE 1991 | Question: 09b

For the following code, indicate the output if 7

a. static scope rules 8
b. dynamic scope rules

are used 9

```
var a,b : integer;

procedure P;
   a := 5;
   b := 10;
end {P};

procedure Q;
   var a, b : integer;
   P;
end {Q};

begin
   a := 1;
   b := 2;
   Q;
   Write ('a = ', a, 'b = ', b);
end
```

10

gate1991 runtime-environment normal compiler-design parameter-passing descriptive 11

Answer key

2.21.8 Parameter Passing: GATE CSE 1993 | Question: 26

A stack is used to pass parameters to procedures in a procedure call.

A. If a procedure P has two parameters as described in procedure definition:

```
procedure P (var x :integer; y: integer);
```

and if P is called by ; $P(a, b)$

State precisely in a sentence what is pushed on stack for parameters a and b

B. In the generated code for the body of procedure P, how will the addressing of formal parameters x and y differ?

gate1993 compiler-design parameter-passing runtime-environment normal descriptive

Answer key

2.21.9 Parameter Passing: GATE CSE 1995 | Question: 2.4

What is the value of X printed by the following program?

```
program COMPUTE (input, output);
var X:integer;
procedure FIND (X:real);
   begin
      X:=sqrt(X);
   end;
begin
   X:=2
   FIND(X);
   writeln(X);
end.
```

A. 2 B. $\sqrt{2}$ C. Run time error D. None of the above

gate1995 compiler-design parameter-passing runtime-environment easy

Answer key

2.21.10 Parameter Passing: GATE CSE 1999 | Question: 15

What will be the output of the following program assuming that parameter passing is

i. call by value
ii. call by reference
iii. call by copy restore

```
procedure P{x, y, z};
begin
   y:y+1;
   z: x+x;
end;
begin
   a:= b:= 3;
   P(a+b, a, a);
   Print(a);
end
```

gate1999 parameter-passing normal runtime-environment descriptive

Answer key

2.21.11 Parameter Passing: GATE CSE 2003 | Question: 74

The following program fragment is written in a programming language that allows global variables and does not allow nested declarations of functions.

```
global int i=100, j=5;
void P(x) {
 int i=10;
   print(x+10);
   i=200;
   j=20;
   print (x);
}
main() {P(i+j);}
```

If the programming language uses dynamic scoping and call by name parameter passing mechanism, the values

printed by the above program are 1

A. $115, 220$ B. $25, 220$ C. $25, 15$ D. $115, 105$ 2

gatecse-2003 programming compiler-design parameter-passing runtime-environment normal 3

Answer key

2.21.12 Parameter Passing: GATE CSE 2004 | Question: 2,ISRO2017-54

Consider the following function 4

```
void swap(int a, int b) 5
{
   int temp;
   temp = a;
   a = b;
   b = temp;
}
```

In order to exchange the values of two variables x and y. 6

A. call $swap(x, y)$ 7
B. call $swap(\&x, \&y)$
C. $swap(x, y)$ cannot be used as it does not return any value
D. $swap(x, y)$ cannot be used as the parameters are passed by value

gatecse-2004 compiler-design programming-in-c parameter-passing easy isro2017 runtime-environment 8

Answer key

2.21.13 Parameter Passing: GATE CSE 2016 | Set 1 | Question: 36

What will be the output of the following pseudo-code when parameters are passed by reference and 9
dynamic scoping is assumed?

```
a = 3;
void n(x) { x = x * a; print (x); }
void m(y) { a = 1 ; a = y - a;  n(a); print (a); }
void main () { m(a); }
```
10

A. $6, 2$ B. $6, 6$ C. $4, 2$ D. $4, 4$ 11

gatecse-2016-set1 parameter-passing normal 12

Answer key 13

2.21.14 Parameter Passing: GATE IT 2007 | Question: 33

Consider the program below in a hypothetical language which allows global variable and a choice of call by 14
reference or call by value methods of parameter passing.

```
int i ;
program main ()
{
   int j = 60;
   i = 50;
   call f (i, j);
   print i, j;
}
procedure f (x, y)
{
   i = 100;
   x = 10;
   y = y + i ;
}
```
15

Which one of the following options represents the correct output of the program for the two parameter passing 16
mechanisms?

A. Call by value : $i = 70, j = 10$; Call by reference : $i = 60, j = 70$ 17
B. Call by value : $i = 50, j = 60$; Call by reference : $i = 50, j = 70$

C. Call by value : $i = 10, j = 70$; Call by reference : $i = 100, j = 60$ 1
D. Call by value : $i = 100, j = 60$; Call by reference : $i = 10, j = 70$

gateit-2007 programming parameter-passing normal compiler-design runtime-environment

Answer key

2.22 Parsing (22)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (7Q)

2.22.1 Parsing: GATE CSE 1987 | Question: 1-xiv

An operator precedence parser is a 2

A. Bottom-up parser.
B. Top-down parser. 3
C. Back tracking parser.
D. None of the above.

gate1987 compiler-design parsing easy 4

Answer key

2.22.2 Parsing: GATE CSE 1989 | Question: 1-iii

Merging states with a common core may produce _____ conflicts and does not produce _____ conflicts in an LALR parser. 5

gate1989 descriptive compiler-design parsing 6

Answer key

2.22.3 Parsing: GATE CSE 1993 | Question: 25

A simple Pascal like language has only three statements. 7

i. assignment statement e.g. x:=expression
ii. loop construct e.g. for i:=expression to expression do statement 8
iii. sequencing e.g. begin statement ;…; statement end

A. Write a context-free grammar (CFG) for statements in the above language. Assume that expression has already been defined. Do not use optional parenthesis and * operator in CFG. 9
B. Show the parse tree for the following statements:

```
for j:=2 to 10 do
begin
   x:=expr1;
   y:=expr2;
end
```

10

gate1993 compiler-design parsing normal descriptive 11

Answer key

2.22.4 Parsing: GATE CSE 1995 | Question: 8

Construct the $\mathrm{LL}(1)$ table for the following grammar.

1. $Expr \rightarrow \_Expr$
2. $Expr \rightarrow (Expr)$
3. $Expr \rightarrow Var\ ExprTail$
4. $ExprTail \rightarrow \_Expr$
5. $Expr \rightarrow \lambda$
6. $Var \rightarrow Id\ VarTail$
7. $VarTail \rightarrow (Expr)$
8. $VarTail \rightarrow \lambda$
9. $Goal \rightarrow Expr\$$

gate1995 compiler-design parsing normal descriptive

Answer key

2.22.5 Parsing: GATE CSE 1998 | Question: 1.27

Type checking is normally done during 1

A. lexical analysis
B. syntax analysis 2
C. syntax directed translation
D. code optimization

gate1998 compiler-design parsing easy 3

Answer key

2.22.6 Parsing: GATE CSE 1998 | Question: 22

A. An identifier in a programming language consists of up to six letters and digits of which the first character must be a letter. Derive a regular expression for the identifier. 4

B. Build an $LL(1)$ parsing table for the language defined by the $LL(1)$ grammar with productions

$\text{Program} \rightarrow \text{ begin } d \text{ semi } X \text{ end}$

$X \rightarrow d \text{ semi } X \mid sY$

$Y \rightarrow \text{ semi } sY \mid \epsilon$

gate1998 compiler-design parsing descriptive

Answer key

2.22.7 Parsing: GATE CSE 1999 | Question: 1.17

Which of the following is the most powerful parsing method? 5

A. LL (1) B. Canonical LR C. SLR D. LALR 6

gate1999 compiler-design parsing easy

Answer key

2.22.8 Parsing: GATE CSE 2000 | Question: 1.19, UGCNET-Dec2013-II: 30

Which of the following derivations does a top-down parser use while parsing an input string? The input is 8 scanned from left to right.

A. Leftmost derivation
B. Leftmost derivation traced out in reverse 9
C. Rightmost derivation
D. Rightmost derivation traced out in reverse

gatecse-2000 compiler-design parsing normal ugcnetcse-dec2013-paper2 10

Answer key

2.22.9 Parsing: GATE CSE 2001 | Question: 16

Consider the following grammar with terminal alphabet $\Sigma = \{a, (,), +, *\}$ and start symbol E. The production rules of the grammar are:

- $E \rightarrow aA$
- $E \rightarrow (E)$
- $A \rightarrow +E$
- $A \rightarrow *E$
- $A \rightarrow \epsilon$

a. Compute the FIRST and FOLLOW sets for E and A.
b. Complete the LL(1) parse table for the grammar.

gatecse-2001 compiler-design parsing normal descriptive

Answer key

2.22.10 Parsing: GATE CSE 2003 | Question: 16

Which of the following suffices to convert an arbitrary CFG to an LL(1) grammar? 1

A. Removing left recursion alone
B. Factoring the grammar alone 2
C. Removing left recursion and factoring the grammar
D. None of the above

gatecse-2003 compiler-design parsing easy 3

Answer key

2.22.11 Parsing: GATE CSE 2005 | Question: 83a

Statement for Linked Answer Questions 83a & 83b: 4

Consider the following expression grammar. The semantic rules for expression evaluation are stated next to each grammar production. 5

$$\begin{array}{l|l} E \rightarrow number & E.val = number.val \\ \quad | \; E \text{ `}+\text{'} \; E & E^{(1)}.val = E^{(2)}.val + E^{(3)}.val \\ \quad | \; E \text{ `}\times\text{'} \; E & E^{(1)}.val = E^{(2)}.val \times E^{(3)}.val \end{array}$$ 6

The above grammar and the semantic rules are fed to a *yaac* tool (which is an LALR(1) parser generator) for parsing and evaluating arithmetic expressions. Which one of the following is true about the action of *yaac* for the given grammar? 7

A. It detects *recursion* and eliminates recursion 8
B. It detects *reduce-reduce* conflict, and resolves
C. It detects *shift-reduce* conflict, and resolves the conflict in favor of a *shift* over a *reduce* action
D. It detects *shift-reduce* conflict, and resolves the conflict in favor of a *reduce* over a *shift* action

gatecse-2005 compiler-design parsing difficult

Answer key

2.22.12 Parsing: GATE CSE 2005 | Question: 83b

Consider the following expression grammar. The semantic rules for expression evaluation are stated next to each grammar production. 9

$$\begin{array}{l|l} E \rightarrow number & E.val = number.val \\ \quad | \; E \text{ `}+\text{'} \; E & E^{(1)}.val = E^{(2)}.val + E^{(3)}.val \\ \quad | \; E \text{ `}\times\text{'} \; E & E^{(1)}.val = E^{(2)}.val \times E^{(3)}.val \end{array}$$ 10

Assume the conflicts of this question are resolved using yacc tool and an LALR(1) parser is generated for parsing arithmetic expressions as per the given grammar. Consider an expression $3 \times 2 + 1$. What precedence and associativity properties does the generated parser realize? 11

A. Equal precedence and left associativity; expression is evaluated to 7 12
B. Equal precedence and right associativity; expression is evaluated to 9
C. Precedence of '$\times$' is higher than that of '$+$', and both operators are left associative; expression is evaluated to 7
D. Precedence of '$+$' is higher than that of '$\times$', and both operators are left associative; expression is evaluated to 9

gatecse-2005 compiler-design parsing normal

Answer key

2.22.13 Parsing: GATE CSE 2006 | Question: 58

Consider the following grammar: 1

- $S \rightarrow FR$ 2
- $R \rightarrow *S \mid \varepsilon$
- $F \rightarrow id$

In the predictive parser table M of the grammar the entries $M[S, id]$ and $M[R, \$]$ respectively are 3

A. $\{S \rightarrow FR\}$ and $\{R \rightarrow \varepsilon\}$ 4
B. $\{S \rightarrow FR\}$ and $\{\}$
C. $\{S \rightarrow FR\}$ and $\{R \rightarrow *S\}$
D. $\{F \rightarrow id\}$ and $\{R \rightarrow \varepsilon\}$

gatecse-2006 compiler-design parsing normal 5

Answer key

2.22.14 Parsing: GATE CSE 2007 | Question: 18

Which one of the following is a top-down parser? 6

A. Recursive descent parser. B. Operator precedence parser. 7
C. An LR(k) parser. D. An LALR(k) parser.

gatecse-2007 compiler-design parsing normal 8

Answer key

2.22.15 Parsing: GATE CSE 2008 | Question: 11 9

Which of the following describes a handle (as applicable to LR-parsing) appropriately? 10

A. It is the position in a sentential form where the next shift or reduce operation will occur 11
B. It is non-terminal whose production will be used for reduction in the next step
C. It is a production that may be used for reduction in a future step along with a position in the sentential form where the next shift or reduce operation will occur
D. It is the production p that will be used for reduction in the next step along with a position in the sentential form where the right hand side of the production may be found

gatecse-2008 compiler-design parsing normal 12

Answer key

2.22.16 Parsing: GATE CSE 2009 | Question: 42

Which of the following statements are TRUE? 13

I. There exist parsing algorithms for some programming languages whose complexities are less than $\Theta(n^3)$ 14
II. A programming language which allows recursion can be implemented with static storage allocation.
III. No L-attributed definition can be evaluated in the framework of bottom-up parsing.
IV. Code improving transformations can be performed at both source language and intermediate code level.

A. I and II B. I and IV C. III and IV D. I, III and IV 15

gatecse-2009 compiler-design parsing normal 16

Answer key

2.22.17 Parsing: GATE CSE 2012 | Question: 53

For the grammar below, a partial $LL(1)$ parsing table is also presented along with the grammar. Entries that

need to be filled are indicated as $E1, E2,$ and $E3$. ε is the empty string, \$ indicates end of input, and, | separates alternate right hand sides of productions. 1

- $S \rightarrow aAbB \mid bAaB \mid \varepsilon$ 2
- $A \rightarrow S$
- $B \rightarrow S$

	a	b	$
S	E1	E2	$S \rightarrow \varepsilon$
A	$A \rightarrow S$	$A \rightarrow S$	error
B	$B \rightarrow S$	$B \rightarrow S$	$E3$

3

The appropriate entries for $E1, E2,$ and $E3$ are 4

A. $E1 : S \rightarrow aAbB, A \rightarrow S$
$E2 : S \rightarrow bAaB, B \rightarrow S$
$E3 : B \rightarrow S$

B. $E1 : S \rightarrow aAbB, S \rightarrow \varepsilon$ 5
$E2 : S \rightarrow bAaB, S \rightarrow \varepsilon$
$E3 : S \rightarrow \varepsilon$

C. $E1 : S \rightarrow aAbB, S \rightarrow \varepsilon$
$E2 : S \rightarrow bAaB, S \rightarrow \varepsilon$
$E3 : B \rightarrow S$

D. $E1 : A \rightarrow S, S \rightarrow \varepsilon$
$E2 : B \rightarrow S, S \rightarrow \varepsilon$
$E3 : B \rightarrow S$

normal gatecse-2012 compiler-design parsing

Answer key

2.22.18 Parsing: GATE CSE 2015 | Set 3 | Question: 31

Consider the following grammar *G* 6

$S \rightarrow F \mid H$ 7
$F \rightarrow p \mid c$
$H \rightarrow d \mid c$

Where S, F, and H are non-terminal symbols, $p, d,$ and c are terminal symbols. Which of the following statement(s) is/are correct? 8

S1: LL(1) can parse all strings that are generated using grammar *G* 9

S2: LR(1) can parse all strings that are generated using grammar *G*

A. Only S1 B. Only S2 C. Both S1 and S2 D. Neither S1 and S2 10

gatecse-2015-set3 compiler-design parsing normal 11

Answer key 12

2.22.19 Parsing: GATE CSE 2017 | Set 1 | Question: 43

Consider the following grammar: 13

- stmt $\rightarrow$ **if** expr **then** expr **else** expr; stmt | Ò 14
- expr $\rightarrow$ term **relop** term | term
- term $\rightarrow$ id | number
- id $\rightarrow$ **a** | **b** | **c**
- number $\rightarrow [0-9]$

where **relop** is a relational operator (e.g., $<, >, \ldots$), Ò refers to the empty statement, and **if**, **then**, **else** are terminals. 15
Consider a program P following the above grammar containing ten **if** terminals. The number of control flow paths in P is_____ . For example. the program
if e_1 **then** e_2 **else** e_3
has 2 control flow paths. $e_1 \rightarrow e_2$ and $e_1 \rightarrow e_3$.

gatecse-2017-set1 compiler-design parsing normal numerical-answers

Answer key

2.22.20 Parsing: GATE CSE 2024 | Set 1 | Question: 16

Which of the following is/are Bottom-Up Parser(s)? 1

A. Shift-reduce Parser
B. Predictive Parser 2
C. LL(1) Parser
D. LR Parser

gatecse-2024-set1 multiple-selects compiler-design parsing easy one-mark 3

Answer key

2.22.21 Parsing: GATE IT 2005 | Question: 83b

Consider the context-free grammar 4

- $E \rightarrow E + E$ 5
- $E \rightarrow (E * E)$
- $E \rightarrow \text{id}$

where E is the starting symbol, the set of terminals is $\{id, (, +,), *\}$, and the set of non-terminals is $\{E\}$. 6
For the terminal string $id + id + id + id$, how many parse trees are possible?

A. 5 B. 4 C. 3 D. 2 7

gateit-2005 compiler-design parsing normal 8

Answer key 9

2.22.22 Parsing: GATE IT 2008 | Question: 79

A CFG G is given with the following productions where S is the start symbol, A is a non-terminal and a and b are terminals. 10

- $S \rightarrow aS \mid A$ 11
- $A \rightarrow aAb \mid bAa \mid \epsilon$

For the string "$aabbaab$" how many steps are required to derive the string and how many parse trees are there? 12

A. 6 and 1 B. 6 and 2 C. 7 and 2 D. 4 and 2 13

gateit-2008 compiler-design parsing normal 14

Answer key 15

2.23 16 Register Allocation (6) 17

✍ **Practice Test:** Test 1 (7Q) 18

2.23.1 Register Allocation: GATE CSE 1997 | Question: 4.9 19

The expression $(a * b) * c\ op \ldots$ 20
where 'op' is one of '$+$', '$*$' and '$\uparrow$' (exponentiation) can be evaluated on a CPU with single register without storing the value of $(a * b)$ if

A. 'op' is '$+$' or '$*$'
B. 'op' is '$\uparrow$' or '$*$' 21
C. 'op' is '$\uparrow$' or '$+$'
D. not possible to evaluate without storing

gate1997 compiler-design register-allocation normal 22

Answer key

2.23.2 Register Allocation: GATE CSE 2004 | Question: 10

Consider the grammar rule $E \rightarrow E1 - E2$ for arithmetic expressions. The code generated is targeted to a CPU having a single user register. The subtraction operation requires the first operand to be in the register.

If $E1$ and $E2$ do not have any common sub expression, in order to get the shortest possible code 1

A. $E1$ should be evaluated first
B. $E2$ should be evaluated first
C. Evaluation of $E1$ and $E2$ should necessarily be interleaved
D. Order of evaluation of $E1$ and $E2$ is of no consequence 2

gatecse-2004 compiler-design register-allocation normal

Answer key

2.23.3 Register Allocation: GATE CSE 2010 | Question: 37

The program below uses six temporary variables a, b, c, d, e, f. 3

```
a = 1
b = 10
c = 20
d = a + b
e = c + d
f = c + e
b = c + e
e = b + f
d = 5 + e
return d + f
```

4

Assuming that all operations take their operands from registers, what is the minimum number of registers needed to execute this program without spilling? 5

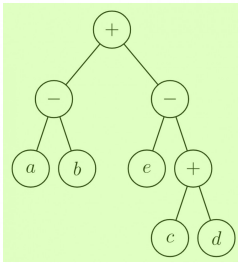
A. 2 B. 3 C. 4 D. 6 6

gatecse-2010 compiler-design register-allocation normal 7

Answer key 8

2.23.4 Register Allocation: GATE CSE 2011 | Question: 36

Consider evaluating the following expression tree on a machine with load-store architecture in which memory can be accessed only through load and store instructions. The variables a, b, c, d, and e are initially stored in memory. The binary operators used in this expression tree can be evaluated by the machine only when operands are in registers. The instructions produce result only in a register. If no intermediate results can be stored in memory, what is the minimum number of registers needed to evaluate this expression? 9



10

A. 2 B. 9 C. 5 D. 3 11

gatecse-2011 compiler-design register-allocation normal 12

Answer key

2.23.5 Register Allocation: GATE CSE 2013 | Question: 48

The following code segment is executed on a processor which allows only register operands in its instructions. Each instruction can have atmost two source operands and one destination operand. Assume that all variables are dead after this code segment.

```
c = a + b;
d = c * a;
e = c + a;
x = c * c;
if (x > a) {
 y = a * a;
```

```
}
else {
 d = d * d;
 e = e * e;
}
```

Q.48 Suppose the instruction set architecture of the processor has only two registers. The only allowed compiler optimization is code motion, which moves statements from one place to another while preserving correctness. What is the minimum number of spills to memory in the compiled code?

A. 0 B. 1 C. 2 D. 3

gatecse-2013 normal compiler-design register-allocation

Answer key

2.23.6 Register Allocation: GATE CSE 2017 | Set 1 | Question: 52

Consider the expression $(a-1)*(((b+c)/3)+d)$. Let X be the minimum number of registers required by an *optimal* code generation (without any register spill) algorithm for a load/store architecture, in which

A. only load and store instructions can have memory operands and
B. arithmetic instructions can have only register or immediate operands.

The value of X is _____ .

gatecse-2017-set1 compiler-design register-allocation normal numerical-answers

Answer key

2.24 Runtime Environment (22)

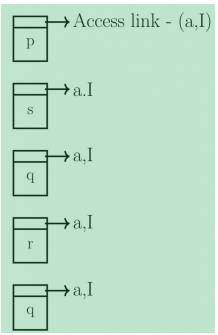
Practice Tests: Test 1 (15Q) Test 2 (2Q)

2.24.1 Runtime Environment: GATE CSE 1988 | Question: 2xii

Consider the following program skeleton and below figure which shows activation records of procedures involved in the calling sequence.

$p \rightarrow s \rightarrow q \rightarrow r \rightarrow q.$

Write the access links of the activation records to enable correct access and variables in the procedures from other procedures involved in the calling sequence




```
procedure p;
 procedure q;
  procedure r;
   begin
    q
   end r;
  begin
   r
  end q;
 procedure s;
  begin
   q
  end s;
 begin
  s
```

```
    end p;
```

gate1988 normal descriptive runtime-environment compiler-design

Answer key

2.24.2 Runtime Environment: GATE CSE 1989 | Question: 10a

Will recursion work correctly in a language with static allocation of all variables? Explain. 1

gate1989 descriptive compiler-design runtime-environment 2

Answer key

2.24.3 Runtime Environment: GATE CSE 1989 | Question: 8b

Indicate the result of the following program if the language uses (i) static scope rules and (ii) dynamic scope rules. 3

```
    var x, y:integer;
    procedure A (var z:integer);
    var x:integer;
    begin x:=1; B; z:= x end;
    procedure B;
    begin x:=x+1 end;
    begin
    x:=5; A(y); write (y)
...end.
```
4

gate1989 descriptive compiler-design runtime-environment 5

Answer key

2.24.4 Runtime Environment: GATE CSE 1990 | Question: 2-v

Match the pairs in the following questions: 6

(a)	Pointer data type	(p)	Type conversion
(b)	Activation record	(q)	Dynamic data structure
(c)	Repeat-until	(r)	Recursion
(d)	Coercion	(s)	Nondeterministic loop

7

gate1990 match-the-following compiler-design runtime-environment recursion

Answer key

2.24.5 Runtime Environment: GATE CSE 1990 | Question: 4-v

State whether the following statements are TRUE or FALSE with reason: 8

The Link-load-and-go loading scheme required less storage space than the link-and-go loading scheme. 9

gate1990 true-false compiler-design runtime-environment 10

Answer key

2.24.6 Runtime Environment: GATE CSE 1993 | Question: 7.7

A part of the system software which under all circumstances must reside in the main memory is:

A. text editor B. assembler C. linker D. loader

E. none of the above

gate1993 compiler-design runtime-environment easy

Answer key

2.24.7 Runtime Environment: GATE CSE 1995 | Question: 1.14

A linker is given object modules for a set of programs that were compiled separately. What information need not be included in an object module?

A. Object code
B. Relocation bits
C. Names and locations of all external symbols defined in the object module
D. Absolute addresses of internal symbols

gate1995 compiler-design runtime-environment normal

Answer key

2.24.8 Runtime Environment: GATE CSE 1996 | Question: 2.17

The correct matching for the following pairs is

(A)	Activation record	(1)	Linking loader
(B)	Location counter	(2)	Garbage collection
(C)	Reference counts	(3)	Subroutine call
(D)	Address relocation	(4)	Assembler

A. A-3 B-4 C-1 D-2 B. A-4 B-3 C-1 D-2 C. A-4 B-3 C-2 D-1 D. A-3 B-4 C-2 D-1

gate1996 compiler-design easy runtime-environment

Answer key

2.24.9 Runtime Environment: GATE CSE 1997 | Question: 1.10

Heap allocation is required for languages.

A. that support recursion
B. that support dynamic data structure
C. that use dynamic scope rules
D. None of the above

gate1997 compiler-design easy runtime-environment

Answer key

2.24.10 Runtime Environment: GATE CSE 1997 | Question: 1.8

A language L allows declaration of arrays whose sizes are not known during compilation. It is required to make efficient use of memory. Which one of the following is true?

A. A compiler using static memory allocation can be written for L
B. A compiler cannot be written for L; an interpreter must be used
C. A compiler using dynamic memory allocation can be written for L
D. None of the above

gate1997 compiler-design easy runtime-environment

Answer key

2.24.11 Runtime Environment: GATE CSE 1998 | Question: 1.25, ISRO2008-41

In a resident – OS computer, which of the following systems must reside in the main memory under all situations?

A. Assembler B. Linker C. Loader D. Compiler

gate1998 compiler-design runtime-environment normal isro2008

Answer key

2.24.12 Runtime Environment: GATE CSE 1998 | Question: 1.28

A linker reads four modules whose lengths are $200, 800, 600$ and 500 words, respectively. If they are loaded in that order, what are the relocation constants? 1

A. $0, 200, 500, 600$
B. $0, 200, 1000, 1600$ 2
C. $200, 500, 600, 800$
D. $200, 700, 1300, 2100$

gate1998 compiler-design runtime-environment normal 3

Answer key

2.24.13 Runtime Environment: GATE CSE 1998 | Question: 2.15

Faster access to non-local variables is achieved using an array of pointers to activation records called a 4

A. stack B. heap C. display D. activation tree 5

gate1998 programming compiler-design normal runtime-environment 6

Answer key

2.24.14 Runtime Environment: GATE CSE 2001 | Question: 1.17

The process of assigning load addresses to the various parts of the program and adjusting the code and the 7 data in the program to reflect the assigned addresses is called

A. Assembly B. parsing C. Relocation D. Symbol resolution 8

gatecse-2001 compiler-design runtime-environment easy 9

Answer key

2.24.15 Runtime Environment: GATE CSE 2002 | Question: 2.20

Dynamic linking can cause security concerns because 10

A. Security is dynamic
B. The path for searching dynamic libraries is not known till runtime
C. Linking is insecure
D. Cryptographic procedures are not available for dynamic linking

11

gatecse-2002 compiler-design runtime-environment easy 12

Answer key

2.24.16 Runtime Environment: GATE CSE 2008 | Question: 54

Which of the following are true? 13

I. A programming language which does not permit global variables of any kind and has no nesting of procedures/functions, but permits recursion can be implemented with static storage allocation 14
II. Multi-level access link (or display) arrangement is needed to arrange activation records only if the programming language being implemented has nesting of procedures/functions
III. Recursion in programming languages cannot be implemented with dynamic storage allocation
IV. Nesting procedures/functions and recursion require a dynamic heap allocation scheme and cannot be implemented with a stack-based allocation scheme for activation records
V. Programming languages which permit a function to return a function as its result cannot be implemented with a stack-based storage allocation scheme for activation records

A. II and V only B. I, III and IV only C. I, II and V only D. II, III and V only 15

gatecse-2008 compiler-design difficult runtime-environment

Answer key

2.24.17 Runtime Environment: GATE CSE 2010 | Question: 14

Which languages necessarily need heap allocation in the runtime environment? 1

A. Those that support recursion.
B. Those that use dynamic scoping. 2
C. Those that allow dynamic data structure.
D. Those that use global variables.

gatecse-2010 compiler-design easy runtime-environment

Answer key

2.24.18 Runtime Environment: GATE CSE 2012 | Question: 36 3

Consider the program given below, in a block-structured pseudo-language with lexical scoping and nesting of procedures permitted. 4

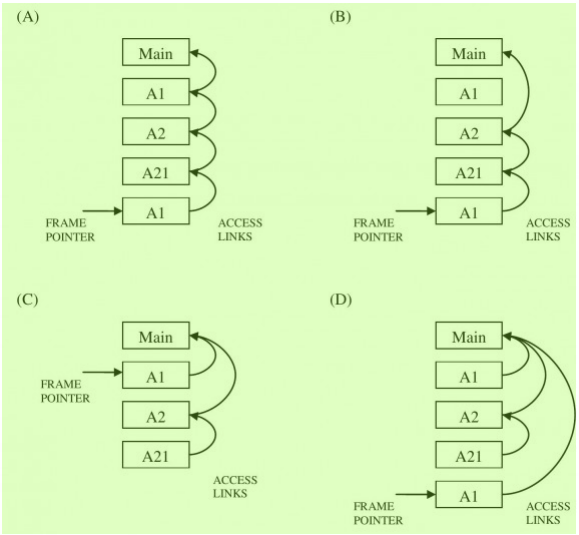
```
Program main;
 Var ...

 Procedure A1;
  Var ...
  Call A2;
 End A1

 Procedure A2;
  Var ...

  Procedure A21;
   Var ...
   Call A1;
  End A21

  Call A21;
 End A2

 Call A1;
End main.
```

5

Consider the calling chain: $\text{Main} \rightarrow \text{A1} \rightarrow \text{A2} \rightarrow \text{A21} \rightarrow \text{A1}$ 6

The correct set of activation records along with their access links is given by: 7

8



gatecse-2012 compiler-design runtime-environment normal

Answer key

2.24.19 Runtime Environment: GATE CSE 2014 | Set 2 | Question: 18

Which one of the following is **NOT** performed during compilation?

A. Dynamic memory allocation
B. Type checking
C. Symbol table management
D. Inline expansion

gatecse-2014-set2 compiler-design easy runtime-environment

Answer key

2.24.20 Runtime Environment: GATE CSE 2014 | Set 3 | Question: 18

Which of the following statements are CORRECT? 1

1. Static allocation of all data areas by a compiler makes it impossible to implement recursion. 2
2. Automatic garbage collection is essential to implement recursion.
3. Dynamic allocation of activation records is essential to implement recursion.
4. Both heap and stack are essential to implement recursion.

A. 1 and 2 only B. 2 and 3 only C. 3 and 4 only D. 1 and 3 only 3

gatecse-2014-set3 compiler-design runtime-environment normal 4

Answer key

2.24.21 Runtime Environment: GATE CSE 2021 | Set 1 | Question: 4

5

Consider the following statements. 6

- S_1 : The sequence of procedure calls corresponds to a preorder traversal of the activation tree. 7
- S_2 : The sequence of procedure returns corresponds to a postorder traversal of the activation tree.

Which one of the following options is correct? 8

A. S_1 is true and S_2 is false
B. S_1 is false and S_2 is true 9
C. S_1 is true and S_2 is true
D. S_1 is false and S_2 is false

gatecse-2021-set1 runtime-environment normal one-mark 10

Answer key

2.24.22 Runtime Environment: GATE CSE 2023 | Question: 26

Consider the following program: 11

12

```
int main()
{
   f1 ();
   f2(2);
   f3();
   return (0);
}
```

```
int f1 ()
{
   return(1);
}
```

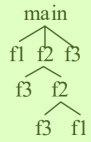
```
int f2 (int X)
{
   f3();
   if (X==1);
      return f1 ();
   else
      return (X * f2 (X - 1));

}
```

```
int f3 ()
{
   return (5);
}
```

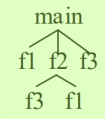
Which one of the following options represents the activation tree corresponding to the main function? 13

14

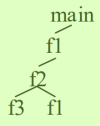
A.



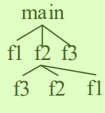
B.



C.



D.



gatecse-2023 compiler-design runtime-environment two-marks 15

Answer key 16

2.25 Static Single Assignment (3)

✍ **Practice Test:** Test 1 (6Q)

2.25.1 Static Single Assignment: GATE CSE 2015 | Set 1 | Question: 55

The least number of temporary variables required to create a three-address code in static single assignment form for the expression $q + r/3 + s - t * 5 + u * v/w$ is_____.

gatecse-2015-set1 compiler-design intermediate-code normal numerical-answers static-single-assignment

Answer key

2.25.2 Static Single Assignment: GATE CSE 2016 | Set 1 | Question: 19

Consider the following code segment.

```
x = u - t;
y = x * v;
x = y + w;
y = t - z;
y = x * y;
```

The minimum number of *total* variables required to convert the above code segment to *static single assignment* form is _____.

gatecse-2016-set1 compiler-design static-single-assignment normal numerical-answers

Answer key

2.25.3 Static Single Assignment: GATE CSE 2017 | Set 1 | Question: 12

Consider the following intermediate program in three address code

```
p = a - b
q = p * c
p = u * v
q = p + q
```

Which one of the following corresponds to a *static single assignment* form of the above code?

A.		B.		C.		D.	
	p1 = a - b		p3 = a - b		p1 = a - b		p1 = a - b
	q1 = p1 * c		q4 = p3 * c		q1 = p2 * c		q1 = p * c
	p1 = u * v		p4 = u * v		p3 = u * v		p2 = u * v
	q1 = p1 + q1		q5 = p4 + q4		q2 = p4 + q3		q2 = p + q

gatecse-2017-set1 compiler-design intermediate-code normal static-single-assignment

Answer key

2.26 Symbol Table (1)

2.26.1 Symbol Table: GATE CSE 2025 | Set 1 | Question: 2

Which ONE of the following statements is **FALSE** regarding the symbol table?

A. Symbol table is responsible for keeping track of the scope of variables.
B. Symbol table can be implemented using a binary search tree.
C. Symbol table is not required after the parsing phase.
D. Symbol table is created during the lexical analysis phase.

gatecse2025-set1 compiler-design symbol-table easy one-mark

Answer key

2.27 Syntax Directed Translation (19)

Practice Tests: Test 1 (15Q) Test 2 (10Q)

2.27.1 Syntax Directed Translation: GATE CSE 1992 | Question: 11a

Write syntax directed definitions (semantic rules) for the following grammar to add the type of each identifier to its entry in the symbol table during semantic analysis. Rewriting the grammar is not permitted and semantic rules are to be added to the ends of productions only.

- $D \rightarrow TL;$
- $T \rightarrow \text{int}$
- $T \rightarrow \text{real}$
- $L \rightarrow L, id$
- $L \rightarrow id$

gate1992 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.2 Syntax Directed Translation: GATE CSE 1995 | Question: 2.10

A shift reduce parser carries out the actions specified within braces immediately after reducing with the corresponding rule of grammar

- $S \rightarrow xxW \ \{\text{print}\text{“}1\text{”}\}$
- $S \rightarrow y \ \{\text{print}\text{“}2\text{”}\}$
- $W \rightarrow Sz \ \{\text{print}\text{“}3\text{”}\}$

What is the translation of $xxxyzz$ using the syntax directed translation scheme described by the above rules?

A. 23131 B. 11233 C. 11231 D. 33211

gate1995 compiler-design grammar syntax-directed-translation normal

Answer key

2.27.3 Syntax Directed Translation: GATE CSE 1996 | Question: 20

Consider the syntax-directed translation schema (SDTS) shown below:

- $E \rightarrow E + E$ {print “+”}
- $E \rightarrow E * E$ {print “.”}
- $E \rightarrow id$ {print id.name}
- $E \rightarrow (E)$

An LR-parser executes the actions associated with the productions immediately after a reduction by the corresponding production. Draw the parse tree and write the translation for the sentence.

$(a + b) * (c + d)$, using SDTS given above.

gate1996 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.4 Syntax Directed Translation: GATE CSE 1998 | Question: 23

Let the attribute ‘val’ give the value of a binary number generated by S in the following grammar:

- $S \rightarrow L.L \mid L$
- $L \rightarrow LB \mid B$
- $B \rightarrow 0 \mid 1$

For example, an input 101.101 gives $S.val = 5.625$

Construct a syntax directed translation scheme using only synthesized attributes, to determine $S.val$.

gate1998 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.5 Syntax Directed Translation: GATE CSE 2000 | Question: 19

Consider the syntax directed translation scheme (SDTS) given in the following. Assume attribute evaluation with bottom-up parsing, i.e., attributes are evaluated immediately after a reduction.

- $E \rightarrow E_1 * T \quad \{E.val = E_1.val * T.val\}$
- $E \rightarrow T \quad \{E.val = T.val\}$
- $T \rightarrow F - T_1 \quad \{T.val = F.val - T_1.val\}$
- $T \rightarrow F \quad \{T.val = F.val\}$
- $F \rightarrow 2 \quad \{F.val = 2\}$
- $F \rightarrow 4 \quad \{F.val = 4\}$

A. Using this SDTS, construct a parse tree for the expression $4 - 2 - 4 * 2$ and also compute its $E.val$.
B. It is required to compute the total number of reductions performed to parse a given input. Using synthesized attributes only, modify the SDTS given, without changing the grammar, to find $E.red$, the number of reductions performed while reducing an input to E.

gatecse-2000 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.6 Syntax Directed Translation: GATE CSE 2001 | Question: 17

The syntax of the repeat-until statement is given by the following grammar

$S \rightarrow \text{repeat } S_1 \text{ until } E$

where E stands for expressions, S and S_1 stand for statements. The non-terminals S and S_1 have an attribute code that represents generated code. The non-terminal E has two attributes. The attribute code represents generated code to evaluate the expression and store its value in a distinct variable, and the attribute varName contains the name of the variable in which the truth value is stored. The truth-value stored in the variable is 1 if E is true, 0 if E is false.

Give a syntax-directed definition to generate three-address code for the repeat-until statement. Assume that you can call a function newlabel() that returns a distinct label for a statement. Use the operator '\\' to concatenate two strings and the function gen(s) to generate a line containing the string s.

gatecse-2001 compiler-design syntax-directed-translation normal descriptive

Answer key

2.27.7 Syntax Directed Translation: GATE CSE 2003 | Question: 18

In a bottom-up evaluation of a syntax directed definition, inherited attributes can

A. always be evaluated
B. be evaluated only if the definition is L-attributed
C. be evaluated only if the definition has synthesized attributes
D. never be evaluated

gatecse-2003 compiler-design syntax-directed-translation normal

Answer key

2.27.8 Syntax Directed Translation: GATE CSE 2003 | Question: 59

Consider the syntax directed definition shown below.

$$
\begin{array}{ll}
S \rightarrow \textbf{id} := E & \{gen(\textbf{id}.place = E.place;);\} \\
E \rightarrow E_1 + E_2 & \{t = newtemp(); \\
 & gen(t = E_1.place + E_2.place;); \\
 & E.place = t;\} \\
E \rightarrow id & \{E.place = \textbf{id}.place;\}
\end{array}
$$

Here, gen is a function that generates the output code, and $newtemp$ is a function that returns the name of a new temporary variable on every call. Assume that t_i's are the temporary variable names generated by $newtemp$. For the statement '$X := Y + Z$', the 3-address code sequence generated by this definition is 1

A. $X = Y + Z$
B. $t_1 = Y + Z; X = t_1$
C. $t_1 = Y; t_2 = t_1 + Z; X = t_2$
D. $t_1 = Y; t_2 = Z; t_3 = t_1 + t_2; X = t_3$ 2

gatecse-2003 compiler-design syntax-directed-translation normal 3

Answer key 4

2.27.9 Syntax Directed Translation: GATE CSE 2004 | Question: 45

Consider the grammar with the following translation rules and E as the start symbol 5

$E \rightarrow E_1 \# T$	$\{E.value = E_1.value * T.value\}$
$\quad \mid T$	$\{E.value = T.value\}$
$T \rightarrow T_1 \& F$	$\{T.value = T_1.value + F.value\}$
$\quad \mid F$	$\{T.value = F.value\}$
$F \rightarrow \text{num}$	$\{F.value = num.value\}$

6

Compute E.value for the root of the parse tree for the expression $2 \# 3 \& 5 \# 6 \& 4$ 7

A. 200 B. 180 C. 160 D. 40 8

gatecse-2004 compiler-design grammar normal syntax-directed-translation 9

Answer key 10

2.27.10 Syntax Directed Translation: GATE CSE 2016 | Set 1 | Question: 46

Consider the following Syntax Directed Translation Scheme $(SDTS)$, with non-terminals $\{S, A\}$ and terminals $\{a, b\}$. 11

$S \rightarrow aA \quad \{\text{print } 1\}$
$S \rightarrow a \quad \{\text{print } 2\}$
$A \rightarrow Sb \quad \{\text{print } 3\}$ 12

Using the above $SDTS$, the output printed by a bottom-up parser, for the input aab is: 13

A. 1 3 2
B. 2 2 3
C. 2 3 1
D. syntax error 14

gatecse-2016-set1 compiler-design syntax-directed-translation normal

Answer key

2.27.11 Syntax Directed Translation: GATE CSE 2019 | Question: 36

Consider the following grammar and the semantic actions to support the inherited type declaration attributes. Let X_1, X_2, X_3, X_4, X_5, and X_6 be the placeholders for the non-terminals D, T, L or L_1 in the following table: 15

Production rule	Semantic action
$D \rightarrow TL$	$X_1.\text{type} = X_2.\text{type}$
$T \rightarrow \text{int}$	$T.\text{type} = \text{int}$
$T \rightarrow \text{float}$	$T.\text{type} = \text{float}$
$L \rightarrow L_1, id$	$X_3.\text{type} = X_4.\text{type}$ $\text{addType}(id.\text{entry}, X_5.\text{type})$
$L \rightarrow id$	$\text{addType}(id.\text{entry}, X_6.\text{type})$

16

Which one of the following are appropriate choices for X_1, X_2, X_3 and X_4? 17

A. $X_1 = L,\ X_2 = T,\ X_3 = L_1,\ X_4 = L$ 18

B. $X_1 = T,\ X_2 = L,\ X_3 = L_1,\ X_4 = T$ 1
C. $X_1 = L,\ X_2 = L,\ X_3 = L_1,\ X_4 = T$
D. $X_1 = T,\ X_2 = L,\ X_3 = T,\ X_4 = L_1$

gatecse-2019 compiler-design syntax-directed-translation two-marks

Answer key

2.27.12 Syntax Directed Translation: GATE CSE 2020 | Question: 33

Consider the productions $A \to PQ$ and $A \to XY$. Each of the five non-terminals A, P, Q, X, and Y has two attributes: s is a synthesized attribute, and i is an inherited attribute. Consider the following rules. 2

- Rule 1 : $P.i = A.i + 2,\ Q.i = P.i + A.i$, and $A.s = P.s + Q.s$ 3
- Rule 2 : $X.i = A.i + Y.s$ and $Y.i = X.s + A.i$

Which one of the following is TRUE? 4

A. Both Rule 1 and Rule 2 are L-attributed.
B. Only Rule 1 is L-attributed. 5
C. Only Rule 2 is L-attributed.
D. Neither Rule 1 nor Rule 2 is L-attributed.

gatecse-2020 compiler-design syntax-directed-translation two-marks

Answer key

2.27.13 Syntax Directed Translation: GATE CSE 2021 | Set 1 | Question: 26

Consider the following grammar (that admits a series of declarations, followed by expressions) and the associated syntax directed translation (SDT) actions, given as pseudo-code 6

$$
\begin{array}{lcl}
P & \to & D^*E^* \\
D & \to & \text{int ID}\{\text{record that ID.lexeme is of type int}\} \\
D & \to & \text{bool ID}\{\text{record that ID.lexeme is of type bool}\} \\
E & \to & E_1 + E_2\{\text{check that } E_1.\text{type} = E_2.\text{type} = \text{int; set } E.\text{type} := \text{int}\} \\
E & \to & !E_1\{\text{check that } E_1.\text{type} = \text{bool; set } E.\text{type} := \text{bool}\} \\
E & \to & \text{ID}\{\text{set } E.\text{type} := \text{int}\}
\end{array}
$$ 7

With respect to the above grammar, which one of the following choices is correct? 8

A. The actions can be used to correctly type-check any syntactically correct program 9
B. The actions can be used to type-check syntactically correct integer variable declarations and integer expressions
C. The actions can be used to type-check syntactically correct boolean variable declarations and boolean expressions.
D. The actions will lead to an infinite loop

gatecse-2021-set1 compiler-design syntax-directed-translation two-marks

Answer key

2.27.14 Syntax Directed Translation: GATE CSE 2022 | Question: 55

Consider the following grammar along with translation rules.

$$
\begin{array}{ll}
S \to S_1 \# T & \{S.\text{val} = S_1.\text{val} \ {}^*T.\text{val}\} \\
S \to T & \{S.\text{val} = T.\text{val}\} \\
T \to T_1 \% R & \{T.\text{val} = T_1.\text{val} \div R.\text{val}\} \\
T \to R & \{T.\text{val} = R.\text{val}\} \\
R \to \text{id} & \{R.\text{val} = \text{id.val}\}
\end{array}
$$

Here $\#$ and $\%$ are operators and id is a token that represents an integer and $\mathrm{id}.\mathrm{val}$ represents the corresponding integer value. The set of non-terminals is $\{\mathrm{S}, \mathrm{T}, \mathrm{R}, \mathrm{P}\}$ and a subscripted non-terminal indicates an instance of the non-terminal. 1

Using this translation scheme, the computed value of $S.\mathrm{val}$ for root of the parse tree for the expression $20\#10\%5\#8\%2\%2$ is _____. 2

gatecse-2022 numerical-answers compiler-design syntax-directed-translation two-marks

Answer key

2.27.15 Syntax Directed Translation: GATE CSE 2023 | Question: 50

Consider the syntax directed translation given by the following grammar and semantic rules. Here N, I, F and B are non-terminals. N is the starting non-terminal, and $\#, \mathbf{0}$ and $\mathbf{1}$ are lexical tokens corresponding to input letters "#", "0" and "1", respectively. $X.val$ denotes the synthesized attribute (a numeric value) associated with a non-terminal X. I_1 and F_1 denote occurrences of I and F on the right hand side of a production, respectively. For the tokens $\mathbf{0}$ and $\mathbf{1}, \mathbf{0}.val = 0$ and $\mathbf{1}.val = 1$. 3

$N \to I\#F$	$N.val = I.val + F.val$
$I \to I_1 B$	$I.val = (2I_1.val) + B.val$
$I \to B$	$I.val = \mathrm{B.val}$
$F \to BF_1$	$F.val = \frac{1}{2}(B.val + F_1.val)$
$F \to B$	$\mathrm{F.val} = \frac{1}{2}B.val$
$B \to \mathbf{0}$	$B.val = \mathbf{0}.val$
$B \to \mathbf{1}$	$B.val = \mathbf{1}.val$

4

The value computed by the translation scheme for the input string 5

$$10\#011$$

is _____. (Rounded off to three decimal places) 6

gatecse-2023 compiler-design syntax-directed-translation numerical-answers two-marks

Answer key

2.27.16 Syntax Directed Translation: GATE CSE 2024 | Set 1 | Question: 27

Consider the following syntax-directed definition (SDD). 7

$S \to DHTU$	$\{S.val = D.val + H.val + T.val + U.val;\}$
$D \to$" M"D_1	$\{D.val = 5 + D_1.val;\}$
$D \to \epsilon$	$\{D.val = -5;\}$
$H \to$" L"H_1	$\{H.val = 5^*10 + H_1.val;\}$
$H \to \epsilon$	$\{H.val = -10;\}$
$T \to$"C" T_1	$\{T.val = 5^*100 + T_1.val;\}$
$T \to \epsilon$	$\{T.val = -5;\}$
$U \to$"K"	$\{U.val = 5;\}$

8

Given "MMLK" as the input, which one of the following options is the CORRECT value computed by the SDD (in the attribute $S.val$)? 9

A. 45 B. 50 C. 55 D. 65 10

gatecse-2024-set1 compiler-design syntax-directed-translation two-marks

Answer key

2.27.17 Syntax Directed Translation: GATE CSE 2024 | Set 2 | Question: 19

Which of the following statements is/are FALSE? 1

A. An attribute grammar is a syntax-directed definition (SDD) in which the functions in the semantic rules have no side effects 2
B. The attributes in a L-attributed definition cannot always be evaluated in a depth-first order
C. Synthesized attributes can be evaluated by a bottom-up parser as the input is parsed
D. All L-attributed definitions based on $\mathrm{LR}(1)$ grammar can be evaluated using a bottom-up parsing strategy

gatecse-2024-set2 compiler-design syntax-directed-translation multiple-selects one-mark

Answer key

2.27.18 Syntax Directed Translation: GATE CSE 2025 | Set 2 | Question: 12

Given the following syntax directed translation rules: 3

- Rule 1: $R \rightarrow AB\{B.i = R.i - 1; A.i = B.i; R.i = A.i + 1;\}$ 4
- Rule 2: $P \rightarrow CD\{P.i = C.i + D.i; D.i = C.i + 2;\}$
- Rule 3: $Q \rightarrow EF\{Q.i = E.i + F.i;\}$

Which ONE is the CORRECT option among the following? 5

A. Rule 1 is S-attributed and L-attributed; Rule 2 is S-attributed and not L-attributed; Rule 3 is neither S-attributed nor L-attributed. 6
B. Rule 1 is neither S-attributed not L-attributed; Rule 2 is S-attributed and L-attributed; Rule 3 is S-attributed and L-attributed.
C. Rule 1 is neither S-attributed nor L-attributed; Rule 2 is not S-attributed and is L-attributed; Rule 3 is S-attributed and L-attributed.
D. Rule 1 is S-attributed and not L-attributed; Rule 2 is not S-attributed and is L-attributed; Rule 3 is S-attributed and L-attributed.

gatecse2025-set2 compiler-design syntax-directed-translation one-mark

Answer key

2.27.19 Syntax Directed Translation: GATE CSE 2026 | Set 1 | Question: 43

Consider the following two syntax-directed definitions $\mathrm{SDD1}$ and $\mathrm{SDD2}$ for type declarations. 7

8

SDD1	
Grammar (G1)	**Semantic Rules**
$D \rightarrow T\ V$	$D.type = T.type$ $V.type = T.type$
$T \rightarrow \mathrm{int}$	$T.type = \mathrm{int}$
$T \rightarrow \mathrm{float}$	$T.type = \mathrm{float}$
$V \rightarrow V_1\ \mathrm{id}$	$V_1.type = V.type$ $\mathrm{put(id.entry, V.type)}$
$V \rightarrow \mathrm{id}$	$\mathrm{put(id.entry, V.type)}$

9

SDD2	
Grammar (G2)	**Semantic Rules**
$D \rightarrow D_1\ \mathrm{id}$	$D.type = D_1.type$ $\mathrm{put(id.entry, D\_1.type)}$
$D \rightarrow T\ \mathrm{id}$	$D.type = T.type$ $\mathrm{put(id.entry, T.type)}$

$T \rightarrow \text{int}$	$T.type = \text{int}$
$T \rightarrow \text{float}$	$T.type = \text{float}$

1

D is the start symbol, and int, float and id are the three terminals. The non-terminal V_1 is the same as V and the non-terminal D_1 is the same as D. Here, the subscript is used to differentiate the grammar symbols on the two sides of a production. The function put updates the symbol table with the type information for an identifier. 2

Let P and Q be the languages specified by grammars $\mathrm{G1}$ and $\mathrm{G2}$, respectively. 3
Which of the following statements is/are true?

A. The languages P and Q are the same 4
B. $\mathrm{SDD2}$ is S-attributed and contains only synthesized attributes
C. $\mathrm{SDD1}$ is L-attributed and contains only inherited attributes
D. The specifications of $\mathrm{SDD1}$ and $\mathrm{SDD2}$ are such that the same entries get added to the symbol table

gatecse-2026-set1 compiler-design syntax-directed-translation two-marks multiple-selects

Answer key

2.28 Variable Scope (2)

2.28.1 Variable Scope: GATE CSE 1987 | Question: 1-xix

Study the following program written in a block-structured language: 5

```
Var x, y:interger;
procedure P(n:interger);
begin
    x:=(n+2)/(n-3);
end;

procedure Q
Var x, y:interger;
begin
   x:=3;
   y:=4;
   P(y);
   Write(x)                    __(1)
end;

begin
   x:=7;
   y:=8;
   Q;
Write(x);                      __(2)
end.
```

6

What will be printed by the write statements marked (1) and (2) in the program if the variables are statically scoped? 7

A. $3, 6$ B. $6, 7$ C. $3, 7$ D. None of the above. 8

gate1987 compiler-design variable-scope runtime-environment

Answer key

2.28.2 Variable Scope: GATE CSE 1987 | Question: 1-xx

For the program given below what will be printed by the write statements marked (1) and (2) in the program if the variables are dynamically scoped?

```
Var x, y:interger;
procedure P(n:interger);
begin
    x := (n+2)/(n-3);
end;

procedure Q
Var x, y:interger;
begin
   x:=3;
   y:=4;
```

```
   P(y);
   Write(x);                    __(1)
end;

begin
   x:=7;
   y:=8;
   Q;
   Write(x);                    __(2)
end.
```
1

A. $3,6$ B. $6,7$ C. $3,7$ D. None of the above 2

gate1987 compiler-design variable-scope runtime-environment 3

Answer key 4

2.29 Viable Prefix (1)

2.29.1 Viable Prefix: GATE CSE 2015 | Set 1 | Question: 13

Which one of the following is TRUE at any valid state in shift-reduce parsing? 5

A. Viable prefixes appear only at the bottom of the stack and not inside 6
B. Viable prefixes appear only at the top of the stack and not inside
C. The stack contains only a set of viable prefixes
D. The stack never contains viable prefixes

gatecse-2015-set1 compiler-design parsing normal viable-prefix lr-parser 7

Answer key

Answer Keys 8

2.1.1	C	2.2.1	B	2.2.2	B;C	2.3.1	N/A	2.3.2	TBA
2.3.3	N/A	2.3.4	A;B;C	2.3.5	True	2.3.6	N/A	2.3.7	True
2.3.8	False	2.3.9	B	2.4.1	C	2.5.1	6:6	2.5.2	A
2.6.1	A	2.6.2	D	2.6.3	A	2.6.4	D	2.6.5	7
2.6.6	D	2.6.7	B	2.6.8	D	2.7.1	C	2.7.2	N/A
2.7.3	A	2.7.4	B	2.7.5	C	2.7.6	B	2.7.7	C
2.7.8	B	2.7.9	D	2.7.10	C	2.7.11	B	2.7.12	B
2.7.13	C	2.8.1	13:13	2.9.1	A	2.9.2	6 : 6	2.10.1	A
2.10.2	6	2.11.1	A;B	2.11.2	A	2.11.3	C	2.11.4	31
2.11.5	A	2.11.6	A;D	2.12.1	N/A	2.12.2	N/A	2.12.3	N/A
2.12.4	N/A	2.12.5	N/A	2.12.6	C	2.12.7	C	2.12.8	N/A
2.12.9	N/A	2.12.10	X	2.12.11	N/A	2.12.12	N/A	2.12.13	D
2.12.14	N/A	2.12.15	N/A	2.12.16	N/A	2.12.17	C	2.12.18	A
2.12.19	N/A	2.12.20	D	2.12.21	A	2.12.22	B	2.12.23	B
2.12.24	C	2.12.25	A	2.12.26	D	2.12.27	B	2.12.28	D
2.12.29	D	2.12.30	A	2.12.31	C	2.12.32	C	2.12.33	C
2.12.34	B	2.12.35	C	2.12.36	C	2.12.37	B	2.12.38	A
2.12.39	C	2.12.40	5	2.12.41	A	2.12.42	179	2.12.43	B;C
2.12.44	C;D	2.12.45	A	2.12.46	C	2.12.47	D	2.13.1	N/A
2.13.2	N/A	2.13.3	N/A	2.13.4	C	2.13.5	A	2.13.6	C

9

2.13.7	A	2.13.8	B	2.13.9	D	2.13.10	D	2.13.11	A
2.14.1	B;C;D	2.14.2	C	2.14.3	B	2.14.4	B	2.14.5	D
2.14.6	B	2.14.7	D	2.14.8	B	2.14.9	D	2.14.10	C
2.14.11	A	2.14.12	D	2.14.13	7	2.14.14	C	2.14.15	8 : 8
2.14.16	5	2.14.17	D	2.14.18	9	2.14.19	C	2.14.20	D
2.15.1	A	2.15.2	C	2.15.3	B	2.15.4	C	2.15.5	A
2.15.6	D	2.16.1	B	2.16.2	C	2.16.3	C	2.17.1	C
2.17.2	A	2.17.3	D	2.18.1	A	2.18.2	C	2.19.1	N/A
2.19.2	D	2.19.3	A	2.19.4	D	2.20.1	N/A	2.20.2	A
2.20.3	B	2.20.4	N/A	2.20.5	B	2.20.6	B	2.20.7	9
2.20.8	A	2.20.9	6	2.21.1	10	2.21.2	N/A	2.21.3	A;D
2.21.4	N/A	2.21.5	A;D	2.21.6	N/A	2.21.7	N/A	2.21.8	N/A
2.21.9	A	2.21.10	N/A	2.21.11	X	2.21.12	D	2.21.13	D
2.21.14	D	2.22.1	A	2.22.2	N/A	2.22.3	N/A	2.22.4	N/A
2.22.5	C	2.22.6	N/A	2.22.7	B	2.22.8	A	2.22.9	N/A
2.22.10	D	2.22.11	C	2.22.12	B	2.22.13	A	2.22.14	A
2.22.15	D	2.22.16	B	2.22.17	C	2.22.18	D	2.22.19	1024
2.22.20	A;D	2.22.21	A	2.22.22	A	2.23.1	A	2.23.2	B
2.23.3	B	2.23.4	D	2.23.5	B	2.23.6	2	2.24.1	N/A
2.24.2	N/A	2.24.3	N/A	2.24.4	N/A	2.24.5	True	2.24.6	D
2.24.7	C	2.24.8	D	2.24.9	B	2.24.10	C	2.24.11	C
2.24.12	B	2.24.13	C	2.24.14	C	2.24.15	B	2.24.16	A
2.24.17	C	2.24.18	D	2.24.19	A	2.24.20	D	2.24.21	C
2.24.22	A	2.25.1	8	2.25.2	10	2.25.3	B	2.26.1	C
2.27.1	N/A	2.27.2	A	2.27.3	N/A	2.27.4	N/A	2.27.5	N/A
2.27.6	N/A	2.27.7	B	2.27.8	B	2.27.9	C	2.27.10	C
2.27.11	A	2.27.12	B	2.27.13	B	2.27.14	80	2.27.15	2.375
2.27.16	A	2.27.17	B;D	2.27.18	C	2.27.19	A;B;D	2.28.1	A
2.28.2	B	2.29.1	C						

3 Programming and DS: Data Structures (238)

Webpage 1

Stacks, Queues, Linked lists, Trees, Binary search trees, Binary heaps, Graphs. 2

Mark Distribution in Previous GATE 3

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum 4
1 Mark Count	0	1	3	2	0	0	2	2	4	2	0
2 Marks Count	1	1	1	2	1	2	3	1	1	0	0
Total Marks	2	3	5	6	2	4	8	4	6	2	**2**

Welcome to the "Programming and Data Structures: Data Structures" chapter, a cornerstone of the GATE Computer Science syllabus. This subject is fundamental to understanding how data is organized, stored, and manipulated efficiently, which is crucial for designing optimal algorithms and software systems. For GATE CS, Data Structures typically carries a significant weightage, often contributing 8-12 marks, sometimes more, through a mix of Multiple Choice Questions (MCQs) and Numerical Answer Type (NAT) questions. Questions frequently test your understanding of core concepts, time and space complexity analysis of operations, properties of various data structures, and their applications in problem-solving. Expect problems involving tracing operations on specific data structures like AVL trees or heaps, analyzing the complexity of given code snippets, or determining the output of tree traversals. 5

Topic-wise Key Concepts 6

AVL Tree 7

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between the heights of the left and right subtrees for any node is at most 1. This strict balance ensures that the tree remains relatively balanced, preventing worst-case scenarios of skewed BSTs. 8

- **Balance Factor:** For any node N, $BalanceFactor(N) = Height(RightSubtree(N)) - Height(LeftSubtree(N))$. In an AVL tree, $BalanceFactor(N) \in \{-1, 0, 1\}$.
- **Rotations:** When an insertion or deletion causes a node to become unbalanced (balance factor becomes ± 2), rotations are performed to restore balance. There are four types of rotations:
  1. **LL Rotation (Left-Left):** Occurs when an insertion into the left subtree of the left child causes imbalance. A single right rotation is performed.
  2. **RR Rotation (Right-Right):** Occurs when an insertion into the right subtree of the right child causes imbalance. A single left rotation is performed.
  3. **LR Rotation (Left-Right):** Occurs when an insertion into the right subtree of the left child causes imbalance. A left rotation on the child, followed by a right rotation on the parent.
  4. **RL Rotation (Right-Left):** Occurs when an insertion into the left subtree of the right child causes imbalance. A right rotation on the child, followed by a left rotation on the parent.
- **Height of an AVL Tree:** For an AVL tree with n nodes, its height h is always $O(\log n)$. Specifically, $h < 1.44 \log_2(n+2)$.
- **Time Complexity:** Insertion, deletion, and search operations all take $O(\log n)$ time in the worst case due to the balanced nature. 9

Common Pitfalls: Incorrectly identifying the type of rotation needed, especially for LR and RL cases. Forgetting to update heights and balance factors after rotations. Tracing rotations correctly requires careful step-by-step analysis. 10

Problem-Solving Techniques: Practice tracing insertions/deletions and the subsequent rotations on small example AVL trees. Understand the pivot node and the child involved in the imbalance. 11

Abstract Data Type (ADT) 12

An Abstract Data Type (ADT) is a mathematical model for data types, defining the data stored and the operations that can be performed on it, without specifying how these operations are implemented. It focuses on "what" an operation does rather than "how" it does it. 13

- **Key Properties:**
  - **Abstraction:** Hides the internal implementation details from the user.
  - **Encapsulation:** Bundles data and methods that operate on the data into a single unit. 14
- **Examples:** Stack, Queue, List, Priority Queue, Map, Set are all ADTs. A data structure (e.g., array, linked list) is a

concrete implementation of an ADT.

Common Pitfalls: Confusing an ADT with a data structure. An ADT is a logical concept, while a data structure is a physical implementation.

Problem-Solving Techniques: Identify the core operations and their logical behavior when asked to define or recognize an ADT.

Array

An array is a collection of elements of the same data type, stored in contiguous memory locations. Elements are accessed using an integer index.

- **Address Calculation (1D Array):** For an array A starting at $BaseAddress$, with elements of size S, and lower bound L_0:

$$Address(A[i]) = BaseAddress + (i - L_0) \times S$$

  If $L_0 = 0$, then $Address(A[i]) = BaseAddress + i \times S$.
- **Address Calculation (2D Array - Row-Major Order):** For an array $A[R][C]$ (R rows, C columns), starting at $BaseAddress$, with element size S, and lower bounds L_{row}, L_{col}:

$$Address(A[i][j]) = BaseAddress + ((i - L_{row}) \times C + (j - L_{col})) \times S$$

  If $L_{row} = 0, L_{col} = 0$, then $Address(A[i][j]) = BaseAddress + (i \times C + j) \times S$.
- **Address Calculation (2D Array - Column-Major Order):** For an array $A[R][C]$, starting at $BaseAddress$, with element size S, and lower bounds L_{row}, L_{col}:

$$Address(A[i][j]) = BaseAddress + ((j - L_{col}) \times R + (i - L_{row})) \times S$$

  If $L_{row} = 0, L_{col} = 0$, then $Address(A[i][j]) = BaseAddress + (j \times R + i) \times S$.
- **Time Complexity:**
  - Accessing an element: $O(1)$
  - Insertion/Deletion at end: $O(1)$ (if space available)
  - Insertion/Deletion at beginning/middle: $O(n)$ (requires shifting elements)

Common Pitfalls: Off-by-one errors in index calculations, especially with non-zero lower bounds. Confusing row-major with column-major order.

Problem-Solving Techniques: Carefully apply the address calculation formulas. Understand the memory layout for multi-dimensional arrays.

Binary Heap

A binary heap is a complete binary tree that satisfies the heap property: for a min-heap, every parent node's value is less than or equal to its children's values; for a max-heap, every parent node's value is greater than or equal to its children's values. Heaps are typically implemented using an array.

- **Array Representation (0-indexed):**
  - Parent of node at index i: $\lfloor (i-1)/2 \rfloor$
  - Left child of node at index i: $2i + 1$
  - Right child of node at index i: $2i + 2$
- **Heap Properties:**
  - **Completeness:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Heap Property:** (Min-heap or Max-heap)
- **Time Complexity:**
  - Insert: $O(\log n)$ (heapify-up)
  - Delete-min/max (extract-root): $O(\log n)$ (heapify-down)
  - Build Heap from an array of n elements: $O(n)$
  - Search: $O(n)$ (Heaps are not designed for efficient search of arbitrary elements)

Common Pitfalls: Violating the completeness property. Incorrectly applying heapify-up or heapify-down logic. Confusing min-heap and max-heap properties.

Problem-Solving Techniques: Trace heap operations (insert, delete) step-by-step. For building a heap, start from the last non-leaf node and perform heapify-down upwards.

Binary Search Tree (BST)

A Binary Search Tree (BST) is a binary tree where for every node, all values in its left subtree are less than the node's value, and all values in its right subtree are greater than the node's value. Duplicate values are typically not allowed or handled specifically (e.g., placed in the right subtree).

- **BST Properties:**
  - Left subtree values $<$ Root value
  - Right subtree values $>$ Root value
  - Both left and right subtrees are also BSTs.
  - Inorder traversal of a BST yields elements in sorted order.
- **Time Complexity (Average Case):**
  - Search, Insertion, Deletion: $O(\log n)$
- **Time Complexity (Worst Case - Skewed Tree):**
  - Search, Insertion, Deletion: $O(n)$ (occurs when elements are inserted in sorted or reverse-sorted order, leading to a linked list-like structure)
- **Deletion Cases:**
  1. Node to be deleted is a leaf: Simply remove it.
  2. Node has one child: Replace the node with its child.
  3. Node has two children: Replace the node's value with its inorder predecessor (largest in left subtree) or inorder successor (smallest in right subtree), then delete the predecessor/successor node (which will have 0 or 1 child).

Common Pitfalls: Forgetting the worst-case $O(n)$ complexity for skewed trees. Incorrectly handling deletion of nodes with two children. Not understanding that inorder traversal is key to sorted output.

Problem-Solving Techniques: Practice constructing BSTs and performing deletions. Understand the recursive nature of BST operations.

Binary Tree

A binary tree is a tree data structure in which each node has at most two children, referred to as the left child and the right child.

- **Key Properties:**
  - **Full Binary Tree:** Every node has either 0 or 2 children.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Perfect Binary Tree:** All internal nodes have two children, and all leaf nodes are at the same level. (A perfect binary tree is always full and complete).
  - **Skewed Binary Tree:** All nodes have only one child (either left or right), resembling a linked list.
- **Formulas:**
  - Maximum number of nodes at level k (root at level 0): 2^k
  - Maximum number of nodes in a binary tree of height h (max levels $h+1$): $2^{h+1} - 1$
  - Minimum height of a binary tree with n nodes: $\lceil \log_2(n+1) \rceil - 1$ or $\lfloor \log_2 n \rfloor$ (for complete binary tree)
  - Number of leaf nodes L and internal nodes I in a full binary tree: $L = I + 1$

Common Pitfalls: Confusing the definitions of full, complete, and perfect binary trees. Miscalculating height or number of nodes.

Problem-Solving Techniques: Draw examples to visualize properties. Apply formulas carefully based on the definition of the tree type.

Data Structures

Data structures are specialized formats for organizing and storing data in a computer so that it can be accessed and modified efficiently. They provide a means to manage large amounts of data effectively for various applications.

- **Classification:**
  - **Primitive vs. Non-Primitive:** Primitive (int, char, float), Non-Primitive (Arrays, Linked Lists, Trees, Graphs).
  - **Linear vs. Non-Linear:** Linear (Arrays, Linked Lists, Stacks, Queues), Non-Linear (Trees, Graphs).

    - **Static vs. Dynamic:** Static (fixed size at compile time, e.g., arrays), Dynamic (size can change at runtime, e.g., linked lists).
- **Importance:** Choosing the right data structure is critical for optimal algorithm performance (time and space complexity).

Common Pitfalls: Not understanding the trade-offs between different data structures (e.g., array vs. linked list for insertion/deletion).

Problem-Solving Techniques: Analyze problem requirements (e.g., frequent insertions, fast searches, memory constraints) to select the most appropriate data structure.

Hashing

Hashing is a technique used to map keys to array indices (hash table slots) for efficient data storage and retrieval. A hash function $h(k)$ computes an index for a given key k.

- **Hash Functions:**
    - **Division Method:** $h(k) = k \pmod m$, where m is the size of the hash table (often a prime number not close to a power of 2).
    - **Multiplication Method:** $h(k) = \lfloor m(kA \pmod 1) \rfloor$, where A is a constant $0 < A < 1$ (e.g., $A \approx (\sqrt{5}-1)/2$).
    - **Mid-Square Method:** Square the key, then extract some middle digits.
    - **Folding Method:** Divide the key into parts, then combine them (e.g., add them).
- **Collision Resolution Techniques:** When two different keys map to the same index.
    - **Chaining:** Each hash table slot points to a linked list of all keys that hash to that slot.
        - Average case for search/insert/delete: $O(1+\alpha)$, where α is the load factor.
        - Worst case: $O(n)$ if all keys hash to the same slot.
    - **Open Addressing:** All elements are stored directly in the hash table. When a collision occurs, probe for an alternative empty slot.
        - **Linear Probing:** $h(k,i) = (h'(k) + i) \pmod m$, where $i = 0, 1, 2, \ldots$. Suffers from primary clustering.
        - **Quadratic Probing:** $h(k,i) = (h'(k) + c_1 i + c_2 i^2) \pmod m$. Reduces primary clustering but can suffer from secondary clustering.
        - **Double Hashing:** $h(k,i) = (h_1(k) + i \cdot h_2(k)) \pmod m$. Uses two hash functions to generate probe sequences, reducing clustering.
- **Load Factor (α):** $\alpha = n/m$, where n is the number of elements and m is the table size.
    - For chaining: α can be greater than 1.
    - For open addressing: $\alpha \leq 1$.

Common Pitfalls: Choosing a non-prime table size for division method. Incorrectly applying probing sequences. Understanding the difference between primary and secondary clustering.

Problem-Solving Techniques: Trace insertions and searches with given hash functions and collision resolution strategies. Calculate load factor and analyze performance implications.

Infix, Prefix, Postfix Notations

These are different ways to write arithmetic expressions, differing in the position of operators relative to their operands.

- **Infix Notation:** Operator is between operands (e.g., $A + B$). This is the human-readable form.
- **Prefix Notation (Polish Notation):** Operator precedes its operands (e.g., $+AB$).
- **Postfix Notation (Reverse Polish Notation):** Operator follows its operands (e.g., $AB+$).
- **Properties:** Prefix and Postfix notations do not require parentheses to define operator precedence or associativity, making them unambiguous for computer evaluation.
- **Conversion Rules:**
    - **Infix to Postfix/Prefix:** Typically uses a stack. Operators are pushed onto the stack based on precedence and associativity. Operands are directly appended to the output.
    - **Evaluation of Postfix/Prefix:** Also uses a stack. For postfix, operands are pushed; when an operator is encountered, pop two operands, perform the operation, and push the result. For prefix, scan from right to left.

Common Pitfalls: Incorrectly handling operator precedence and associativity during conversions. Mistakes in stack operations (push/pop order).

Problem-Solving Techniques: Practice conversions using the stack-based algorithms. Remember operator

precedence (e.g., $\times, / > +, -$) and associativity (most are left-associative, exponentiation is right-associative).

Linked List

A linked list is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element (node) contains data and a pointer (or link) to the next node in the sequence.

- **Types of Linked Lists:**
  - **Singly Linked List:** Each node points to the next node. Traversal is unidirectional.
  - **Doubly Linked List:** Each node has pointers to both the next and previous nodes. Traversal is bidirectional.
  - **Circular Linked List:** The last node points back to the first node (or head). Can be singly or doubly circular.
- **Time Complexity:**
  - Insertion/Deletion at beginning: $O(1)$
  - Insertion/Deletion at end: $O(n)$ for singly linked list (requires traversal), $O(1)$ if tail pointer is maintained. $O(1)$ for doubly linked list if tail pointer is maintained.
  - Insertion/Deletion at specific position: $O(n)$ (requires traversal to find position)
  - Search: $O(n)$
- **Advantages:** Dynamic size, efficient insertions/deletions at specific points (if pointer to previous node is available).
- **Disadvantages:** More memory overhead (for pointers), no random access ($O(n)$ for access).

Common Pitfalls: Null pointer exceptions when traversing or manipulating the list. Incorrectly updating pointers during insertion/deletion, leading to lost nodes or broken links. Handling edge cases like empty list or single-node list.

Problem-Solving Techniques: Draw diagrams to visualize pointer changes. Pay close attention to the order of pointer updates. Always check for null pointers.

Priority Queue

A Priority Queue is an Abstract Data Type (ADT) that functions like a queue but with an additional concept of "priority." Elements are retrieved based on their priority, not necessarily their insertion order. The element with the highest (or lowest) priority is always dequeued first.

- **Core Operations:**
  - **Insert (enqueue):** Adds an element with a given priority.
  - **Extract-Min/Max (dequeue):** Removes and returns the element with the highest (or lowest) priority.
  - **Peek:** Returns the highest (or lowest) priority element without removing it.
- **Implementations:**
  - **Binary Heap:** Most efficient implementation, providing $O(\log n)$ for insert and extract-min/max.
  - Unsorted Array/Linked List: $O(1)$ insert, $O(n)$ extract-min/max.
  - Sorted Array/Linked List: $O(n)$ insert, $O(1)$ extract-min/max.

Common Pitfalls: Confusing a priority queue with a regular queue. Not understanding that the underlying data structure (e.g., heap) determines the complexity.

Problem-Solving Techniques: When a problem requires always processing the "most important" item next, a priority queue (implemented with a heap) is often the solution.

Queue

A Queue is a linear Abstract Data Type (ADT) that follows the First-In, First-Out (FIFO) principle. Elements are added at one end (rear/tail) and removed from the other end (front/head).

- **Core Operations:**
  - **Enqueue:** Adds an element to the rear of the queue.
  - **Dequeue:** Removes and returns the element from the front of the queue.
  - **Front/Peek:** Returns the element at the front without removing it.
  - **IsEmpty, IsFull:** Checks the state of the queue.
- **Implementations:**
  - **Array-based:** Can lead to "queue full" even if space is available (linear array). Circular array implementation solves this.
  - **Linked List-based:** More dynamic, no fixed size issues.
- **Time Complexity (Array or Linked List):** All core operations (enqueue, dequeue, front) are $O(1)$.

Common Pitfalls: Underflow (dequeuing from an empty queue) and Overflow (enqueuing into a full array-based queue). Incorrectly managing front and rear pointers in array implementations, especially circular queues.

Problem-Solving Techniques: Trace operations carefully. Understand how front and rear pointers move. For circular queues, remember the modulus operator for index calculations: $(index + 1) \pmod{Size}$.

Stack

A Stack is a linear Abstract Data Type (ADT) that follows the Last-In, First-Out (LIFO) principle. Elements are added and removed only from one end, called the "top."

- **Core Operations:**
  - **Push:** Adds an element to the top of the stack.
  - **Pop:** Removes and returns the element from the top of the stack.
  - **Peek/Top:** Returns the element at the top without removing it.
  - **IsEmpty, IsFull:** Checks the state of the stack.
- **Implementations:**
  - **Array-based:** Uses a fixed-size array and a top pointer/index.
  - **Linked List-based:** Each node points to the next, with the head of the list being the top of the stack.
- **Time Complexity (Array or Linked List):** All core operations (push, pop, peek) are $O(1)$.
- **Applications:** Function call stack, expression evaluation (infix to postfix/prefix conversion), undo/redo functionality, backtracking algorithms.

Common Pitfalls: Underflow (popping from an empty stack) and Overflow (pushing into a full array-based stack). Incorrectly managing the top pointer/index.

Problem-Solving Techniques: Trace stack operations. Recognize problems that naturally fit the LIFO principle (e.g., reversing a sequence, checking parenthesis balance).

Time Complexity

Time complexity measures the amount of time an algorithm takes to run as a function of the input size (n). It's typically expressed using asymptotic notations to describe the growth rate for large n.

- **Asymptotic Notations:**
  - **Big-O Notation (O):** Upper bound. $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Describes the worst-case time.
  - **Big-Omega Notation (Ω):** Lower bound. $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$. Describes the best-case time.
  - **Big-Theta Notation (Θ):** Tight bound. $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Describes average-case time or when best and worst cases are the same.
  - **Little-o Notation (o):** Strict upper bound. $f(n) = o(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$.
  - **Little-omega Notation (ω):** Strict lower bound. $f(n) = \omega(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$.
- **Common Growth Rates (from fastest to slowest):**

$$O(n!), O(2^n), O(n^3), O(n^2), O(n \log n), O(n), O(\log n), O(1)$$

- **Master Theorem for Recurrence Relations:** For recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1, b > 1$ are constants, and $f(n)$ is an asymptotically positive function:
  1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
  2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
  3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, AND if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Common Pitfalls: Ignoring constant factors or lower-order terms (which are dropped in asymptotic analysis). Confusing best, worst, and average case complexities. Incorrectly applying Master Theorem conditions.

Problem-Solving Techniques: Analyze loops (number of iterations). For recursive functions, set up recurrence relations and solve them (often using Master Theorem or substitution method). Identify the dominant term in a sum of complexities.

Tree

A tree is a non-linear, hierarchical data structure consisting of nodes connected by edges. It is a connected acyclic graph, meaning there are no cycles and all nodes are reachable from the root.

- **Key Terminology:**
  - **Root:** The topmost node of the tree.
  - **Parent:** A node that has one or more child nodes.
  - **Child:** A node directly connected to another node when moving away from the root.
  - **Sibling:** Nodes that share the same parent.
  - **Leaf Node (External Node):** A node with no children.
  - **Internal Node:** A node with at least one child.
  - **Ancestors:** All nodes on the path from the root to a node.
  - **Descendants:** All nodes in the subtree rooted at a node.
  - **Depth:** The length of the path from the root to a node (root is at depth 0).
  - **Height:** The length of the longest path from a node to a leaf. The height of the tree is the height of its root.
  - **Degree of a Node:** Number of children it has.
  - **Degree of a Tree:** Maximum degree of any node in the tree.
- **Properties:**
  - A tree with n nodes has exactly $n-1$ edges.
  - There is a unique path between any two nodes in a tree.

Common Pitfalls: Confusing tree terminology (e.g., depth vs. height, parent vs. child). Forgetting the $n-1$ edges property.

Problem-Solving Techniques: Draw trees to visualize concepts. Understand recursive definitions for tree operations.

Tree Traversal

Tree traversal refers to the process of visiting each node in a tree exactly once in a systematic way. There are two main categories: Depth-First Search (DFS) and Breadth-First Search (BFS).

- **Depth-First Search (DFS) Traversal:** Uses a stack (implicitly via recursion) to explore as far as possible along each branch before backtracking.
  - **Inorder Traversal (Left, Root, Right):** Visits the left subtree, then the root, then the right subtree. For BSTs, this yields sorted elements.
  - **Preorder Traversal (Root, Left, Right):** Visits the root, then the left subtree, then the right subtree. Useful for creating a copy of the tree.
  - **Postorder Traversal (Left, Right, Root):** Visits the left subtree, then the right subtree, then the root. Useful for deleting a tree (deletes children before parent).
- **Breadth-First Search (BFS) Traversal (Level Order Traversal):** Visits nodes level by level, from left to right. Uses a queue.
  - Starts at the root, then visits all nodes at depth 1, then all nodes at depth 2, and so on.

Common Pitfalls: Incorrectly applying the order of visits for DFS traversals. Difficulty in reconstructing a tree from given traversals (e.g., Inorder + Preorder can reconstruct a unique BST).

Problem-Solving Techniques: Practice tracing all four traversal types on various trees. Remember the specific order for each. For tree reconstruction, use one traversal (e.g., Preorder) to find the root, then use another (e.g., Inorder) to partition the remaining elements into left and right subtrees.

Uniform Hashing

Uniform Hashing is an idealized assumption used in the analysis of hashing algorithms. It states that each key is equally likely to hash to any slot in the hash table, independently of where other keys hash.

- **Key Property:** If we have n keys and m slots, the probability that a key hashes to any particular slot is $1/m$.
- **Implications:**
  - Minimizes collisions in theory.
  - Leads to the best possible average-case performance for hash tables.
  - Under simple uniform hashing, the expected length of a chain in chaining is $\alpha = n/m$.
  - Under uniform hashing, the expected number of probes for unsuccessful search in open addressing is $1/(1-\alpha)$.

Common Pitfalls: Assuming uniform hashing holds true in practice. Real-world hash functions are rarely perfectly uniform, leading to deviations from theoretical performance.

Problem-Solving Techniques: Understand that uniform hashing is a theoretical model for analyzing the average-case performance of hashing. Use its properties to calculate expected values related to collisions or probes.

Quick Formula Reference

Topic	Formula/Concept
AVL Tree	Balance Factor: $Height(Right) - Height(Left) \in \{-1, 0, 1\}$ Height: $O(\log n)$
Array (1D)	$Address(A[i]) = BaseAddress + (i - L_0) \times S$
Array (2D - Row-Major)	$Address(A[i][j]) = BaseAddress + ((i - L_{row}) \times C + (j - L_{col})) \times S$
Array (2D - Column-Major)	$Address(A[i][j]) = BaseAddress + ((j - L_{col}) \times R + (i - L_{row})) \times S$
Binary Heap (0-indexed)	Parent of i: $\lfloor (i-1)/2 \rfloor$ Left child of i: $2i + 1$ Right child of i: $2i + 2$
Binary Tree	Max nodes at level k: 2^k Max nodes in height h tree: $2^{h+1} - 1$ Min height for n nodes: $\lfloor \log_2 n \rfloor$ (complete tree) Full Binary Tree: $L = I + 1$
Hashing (Division)	$h(k) = k \pmod m$
Hashing (Multiplication)	$h(k) = \lfloor m(kA \pmod 1) \rfloor$
Hashing (Linear Probing)	$h(k, i) = (h'(k) + i) \pmod m$
Hashing (Quadratic Probing)	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod m$
Hashing (Double Hashing)	$h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod m$
Hashing (Load Factor)	$\alpha = n/m$
Time Complexity (Big-O)	$f(n) = O(g(n))$ if $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
Time Complexity (Big-Omega)	$f(n) = \Omega(g(n))$ if $c \cdot g(n) \leq f(n)$ for $n \geq n_0$
Time Complexity (Big-Theta)	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$
Master Theorem (Case 1)	If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$
Master Theorem (Case 2)	If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$
Master Theorem (Case 3)	If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$, then $T(n) = \Theta(f(n))$
Tree (General)	n nodes, $n - 1$ edges

Important Tips for GATE

- **Master Time & Space Complexity:** This is arguably the most crucial aspect. Be able to analyze the complexity of operations on all data structures and for given code snippets. Understand best, worst, and average cases. Practice applying the Master Theorem rigorously.
- **Understand ADT vs. Data Structure:** Clearly differentiate between the abstract concept (ADT) and its concrete implementation (data structure). For example, a Stack is an ADT, while an array-based stack or linked-list based stack are data structures implementing the Stack ADT.
- **Visualize Operations:** For tree-based structures (AVL, BST, Heap) and linked lists, always draw diagrams and trace operations step-by-step. This helps catch errors in pointer manipulation, rotations, or heapify processes.
- **Practice Tree Traversals:** Be proficient in Inorder, Preorder, Postorder, and Level-order traversals. Understand their applications and how to reconstruct a tree from given traversals (e.g., Preorder + Inorder).
- **Hashing Concepts are Key:** Pay close attention to different hash functions, collision resolution techniques (chaining, linear probing, quadratic probing, double hashing), and their impact on performance (load factor, clustering). Be ready to trace insertions and searches.
- **Know Standard Implementations:** Understand how basic ADTs like Stack and Queue are implemented using arrays and linked lists, including edge cases like underflow/overflow and circular array logic.
- **Identify Common Pitfalls:** Be aware of typical mistakes like off-by-one errors in array indexing, null pointer exceptions in linked lists, incorrect balance factor calculations in AVL trees, or misinterpreting operator precedence in expression conversions.
- **Practice Problem-Solving:** Data structures questions are often application-oriented. Practice a wide variety of problems to recognize when to use a particular data structure and how to apply its operations to solve the problem

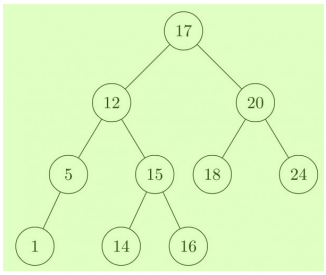
efficiently. 1

3.1 AVL Tree (6)

✍ **Practice Test:** Test 1 (12Q) 2

3.1.1 AVL Tree: GATE CSE 1988 | Question: 7ii 3

Mark the balance factor of each node on the tree given in the below figure and state whether it is height-balanced. 4



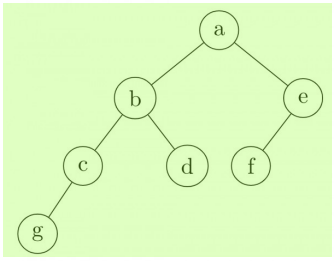
5

gate1988 data-structures normal descriptive avl-tree 6

Answer key

3.1.2 AVL Tree: GATE CSE 1996 | Question: 1.14

In the balanced binary tree in the below figure, how many nodes will become unbalanced when a node is inserted as a child of the node “g”? 7



8

A. 1 B. 3 C. 7 D. 8 9

gate1996 data-structures binary-tree avl-tree normal 10

Answer key

3.1.3 AVL Tree: GATE CSE 1998 | Question: 21

A. Derive a recurrence relation for the size of the smallest AVL tree with height h. 11
B. What is the size of the smallest AVL tree with height 8?

gate1998 data-structures avl-tree descriptive numerical-answers 12

Answer key 13

3.1.4 AVL Tree: GATE CSE 2009 | Question: 37,ISRO-DEC2017-55 14

What is the maximum height of any AVL-tree with 7 nodes? Assume that the height of a tree with a single node is 0. 15

A. 2 B. 3 C. 4 D. 5 16

gatecse-2009 data-structures binary-search-tree normal isrodec2017 avl-tree 17

Answer key

3.1.5 AVL Tree: GATE CSE 2020 | Question: 6

What is the worst case time complexity of inserting n^2 elements into an AVL-tree with n elements initially? 1

A. $\Theta(n^4)$
B. $\Theta(n^2)$ 2
C. $\Theta(n^2 \log n)$
D. $\Theta(n^3)$

gatecse-2020 binary-tree avl-tree one-mark 3

Answer key 4

3.1.6 AVL Tree: GATE IT 2008 | Question: 12

Which of the following is TRUE? 5

A. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a binary search tree is $O(n)$ 6
B. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a complete binary tree is $\Theta(n \log n)$
C. The cost of searching a binary search tree is $O(\log n)$ but that of an AVL tree is $\Theta(n)$
D. The cost of searching an AVL tree is $\Theta(n \log n)$ but that of a binary search tree is $O(n)$

gateit-2008 data-structures binary-search-tree easy avl-tree 7

Answer key

3.2 Abstract Data Type (1)

3.2.1 Abstract Data Type: GATE CSE 2005 | Question: 2

An Abstract Data Type (ADT) is: 8

A. same as an abstract class 9
B. a data type that cannot be instantiated
C. a data type for which only the operations defined on it can be used, but none else
D. all of the above

gatecse-2005 data-structures normal abstract-data-type 10

Answer key 11

3.3 Array (13) 12

Practice Test: Test 1 (9Q) 13

3.3.1 Array: GATE CSE 1993 | Question: 12 14

The following Pascal program segments finds the largest number in a two-dimensional integer array $A[0 \ldots n-1, 0 \ldots n-1]$ using a single loop. Fill up the boxes to complete the program and write against $\boxed{A}, \boxed{B}, \boxed{C}$ and $\boxed{D}$ in your answer book Assume that max is a variable to store the largest value and i, j are the indices to the array. 15

```
begin 16
   max:=|A|, i:=0, j:=0;
   while |B| do
   begin
      if A[i, j]>max then max:=A[i, j];
      if |C| then j:=j+1;
      else begin
         j:=0;
         i:=|D|
      end
   end
end
```

17

gate1993 data-structures array normal descriptive

Answer key

3.3.2 Array: GATE CSE 1994 | Question: 1.11

In a compact single dimensional array representation for lower triangular matrices (i.e all the elements above the diagonal are zero) of size $n \times n$, non-zero elements, (i.e elements of lower triangle) of each row are stored one after another, starting from the first row, the index of the $(i,j)^{th}$ element of the lower triangular matrix in this new representation is:

A. $i+j$ B. $i+j-1$ C. $(j-1)+\frac{i(i-1)}{2}$ D. $i+\frac{j(j-1)}{2}$

gate1994 data-structures array normal

Answer key

3.3.3 Array: GATE CSE 1994 | Question: 25

An array A contains n integers in non-decreasing order, $A[1] \leq A[2] \leq \cdots \leq A[n]$. Describe, using Pascal like pseudo code, a linear time algorithm to find i, j, such that $A[i]+A[j]=a$ given integer M, if such i, j exist.

gate1994 data-structures array normal descriptive

Answer key

3.3.4 Array: GATE CSE 1997 | Question: 17

An array A contains $n \geq 1$ positive integers in the locations $A[1], A[2], \ldots A[n]$. The following program fragment prints the length of a shortest sequence of consecutive elements of A, $A[i], A[i+1], \ldots, A[j]$ such that the sum of their values is $\geq M$, a given positive number. It prints '$n+1$' if no such sequence exists. Complete the program by filling in the boxes. In each case use the simplest possible expression. Write only the line number and the contents of the box.

```
begin
i:=1;j:=1;
sum := □
min:=n; finish:=false;
while not finish do
   if □ then
      if j=n then finish:=true
      else
      begin
         j:=j+1;
         sum:= □
      end
   else
   begin
      if(j-i) < min then min:=j-i;
      sum:=sum –A[i];
      i:=i+1;
   end
   writeln (min +1);
end.
```

gate1997 data-structures array normal descriptive

Answer key

3.3.5 Array: GATE CSE 1998 | Question: 2.14

Let A be a two dimensional array declared as follows:

```
A: array [1 .... 10] [1 ..... 15] of integer;
```

Assuming that each integer takes one memory location, the array is stored in row-major order and the first element of the array is stored at location 100, what is the address of the element $A[i][j]$?

A. $15i+j+84$ B. $15j+i+84$ C. $10i+j+89$ D. $10j+i+89$

gate1998 data-structures array easy

Answer key

3.3.6 Array: GATE CSE 2000 | Question: 1.2

An $n \times n$ array v is defined as follows:
$v[i, j] = i - j$ for all $i, j, i \leq n, 1 \leq j \leq n$
The sum of the elements of the array v is 1

A. 0 B. $n-1$ C. $n^2 - 3n + 2$ D. $n^2 \frac{(n+1)}{2}$ 2

gatecse-2000 data-structures array easy 3

Answer key

3.3.7 Array: GATE CSE 2000 | Question: 15

4

Suppose you are given arrays $p[1.....N]$ and $q[1.....N]$ both uninitialized, that is, each location may 5 contain an arbitrary value), and a variable count, initialized to 0. Consider the following procedures set and is_set:

6

```
set(i) {
    count = count + 1;
    q[count] = i;
    p[i] = count;
}
is_set(i) {
    if (p[i] ≤ 0 or p[i] > count)
        return false;
    if (q[p[i]] ≠ i)
        return false;
    return true;
}
```

7

A. Suppose we make the following sequence of calls:
$set(7); set(3); set(9);$
After these sequence of calls, what is the value of count, and what do $q[1], q[2], q[3], p[7], p[3]$ and $p[9]$ contain?
B. Complete the following statement "The first count elements of _____contain values i such that set (_____) has been called".
C. Show that if $set(i)$ has not been called for some i, then regardless of what $p[i]$ contains, $is\_set(i)$ will return false.

gatecse-2000 data-structures array easy descriptive

Answer key

3.3.8 Array: GATE CSE 2005 | Question: 5

8

A program P reads in 500 integers in the range $[0, 100]$ representing the scores of 500 students. It then prints the frequency of each score above 50. What would be the best way for P to store the frequencies?

9

A. An array of 50 numbers
B. An array of 100 numbers
C. An array of 500 numbers
D. A dynamically allocated array of 550 numbers

gatecse-2005 data-structures array easy 10

Answer key 11

3.3.9 Array: GATE CSE 2013 | Question: 50

12 13

The procedure given below is required to find and replace certain characters inside an input character string supplied in array A. The characters to be replaced are supplied in array $oldc$, while their respective replacement characters are supplied in array $newc$. Array A has a fixed length of five characters, while arrays $oldc$ and $newc$ contain three characters each. However, the procedure is flawed.

14

```
void find_and_replace (char *A, char *oldc, char *newc) {
    for (int i=0; i<5; i++)
        for (int j=0; j<3; j++)
            if (A[i] == oldc[j])
                A[i] = newc[j];
```